
پردازش تصویر در دهه آینده

از عمق ریاضیات تا مهندس حرفه‌ای

کتاب - جزوه درس کارشناسی ارشد و آزمایشگاه پژوهش محور

یازدهم فصل

خودنظارتی یادگیری و بینایی ترنسفورمرهای

سلسله‌مراتبی، های ViT تا مکانی نمایش و توجه جبر تصویر، توکن سازی از
بینایی بنیادی مدل‌های و JEPA ماسک شده، مدل سازی خودتقطیر، InfoNCE

بدانیم دقیقاً باید نیست. شعار یک دنباله» به صورت «تصویر فصل این در
می سازد، عملگری چه توجه می دهد، دست از اطلاعاتی چه توکن هر
کتابخانه‌های و می کند جلوگیری فروپاشی از چگونه خودنظارتی یادگیری
می کنند. پنهان ما برای را مهندسی و ریاضیات از بخش کدام مدرن

نسخه تفصیلی درس - تابستان ۱۴۰۵ / ۲۰۲۶

قابل کامپایل با XeLaTeX + TeXstudio + TeX Live

یادداشت روش شناختی فصل یازدهم

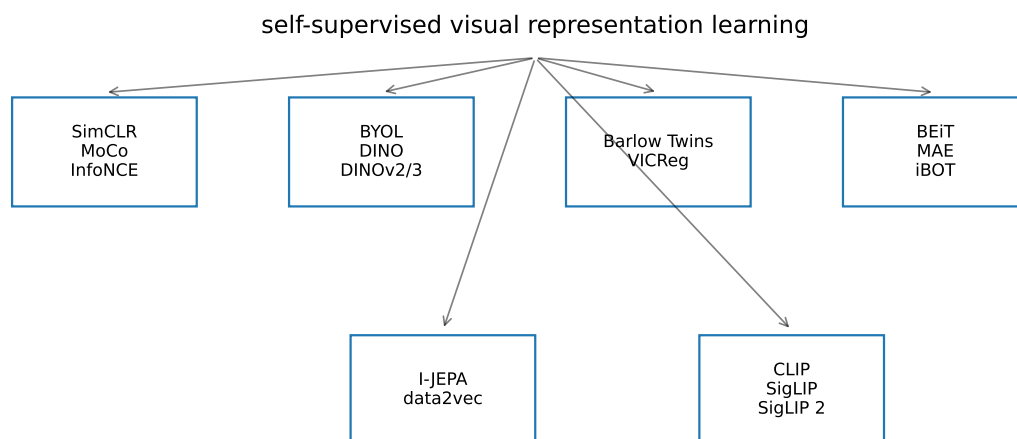
فصل دهم نشان داد که شبکه کانولوشنی با اتصال محلی و اشتراک وزن، یک پیشین هندسی قوی به مدل تحمیل می‌کند. ترنسفورمر بینایی این پیشین را حذف نمی‌کند؛ بلکه آن را نرم‌تر و داده‌محورتر می‌سازد: تصویر ابتدا به مجموعه یا دنباله‌ای از توکن‌ها تبدیل می‌شود و سپس هر توکن می‌تواند، بر اساس محتوای جاری، از توکن‌های دیگر اطلاعات بگیرد. نتیجه، یک عملگر اختلاط وابسته به داده است که ظرفیت مدل‌سازی دوربرد بالایی دارد، اما هزینه محاسباتی، حساسیت به توکن‌سازی، وابستگی به داده و خطر آموختن میان‌برهای مکانی را نیز به همراه می‌آورد.

نیمه دوم فصل به یادگیری خودنظارتی اختصاص دارد. این اصطلاح به معنی «آموزش بدون هیچ سیگنال» نیست؛ سیگنال آموزش از خود داده ساخته می‌شود. در روش‌های کنتراستی، دو نمای مرتبط باید نزدیک و نمونه‌های دیگر دور شوند؛ در خودتقطیری، دانش‌آموز خروجی یک معلم آرام را پیش‌بینی می‌کند؛ در کاهش افزونگی، کوواریانس نمایش کنترل می‌شود؛ در مدل‌سازی ماسک‌شده، بخش‌های پنهان تصویر یا نمایش آن‌ها بازسازی می‌شوند؛ و در معماری‌های پیش‌بینی مشترک، هدف در فضای نهفته پیش‌بینی می‌شود. این فصل همه این خانواده‌ها را با یک زبان ریاضی مشترک مقایسه می‌کند: چه چیزی دیده می‌شود، چه چیزی هدف است، چه ناوردایی تحمیل می‌شود و سازوکار ضد فروپاشی چیست؟

هدف فصل

این فصل پنج خروجی هم‌زمان دارد:

۱. فهم توکن‌سازی: دانشجو بتواند وصله‌سازی را به عنوان یک عملگر خطی تحلیل کند و اثر اندازه وصله، هم‌پوشانی، وضوح و نسبت تصویر را توضیح دهد.
۲. فهم توجه: دانشجو بتواند توجه مقیاس‌شده، چندسری، کدگذاری مکان و پیچیدگی زمانی/حافظه‌ای را استخراج کند.
۳. فهم خودنظارتی: دانشجو بتواند خانواده‌های کنتراستی، خودتقطیری، کاهش افزونگی، ماسک‌شده و پیش‌بینی نهفته را از روی تابع هدف مقایسه کند.
۴. توان کدنویسی: دانشجو بتواند ViT، InfoNCE، MAE و معلم EMA را در نسخه آموزشی از صفر پیاده‌سازی و آزمون کند.
۵. قضاوت پژوهشی و مهندسی: دانشجو بداند کدام مدل و کتابخانه برای طبقه‌بندی، بازیابی، قطعه‌بندی، داده کم‌برچسب، وضوح بومی یا سامانه چندوجهی مناسب‌تر است.



شکل ۱: نقشه خانواده‌های اصلی یادگیری خودنظارتی بینایی. تفاوت بنیادی روش‌ها در فضای هدف، نوع نوردایی و سازوکار جلوگیری از فروپاشی است.

نقشه فشرده فصل

محور	پرسش و خروجی آموزشی
توکن سازی	وصله، توکن هم‌پوشان، توکن سلسله‌مراتبی، توکن تطبیقی و توکن وضوح بومی چه اطلاعاتی را حفظ یا حذف می‌کنند؟
جبر توجه	Q، K و V چگونه ساخته می‌شوند، چرا تقسیم بر $\sqrt{d_k}$ لازم است و گرادیان نرمینه چه رفتاری دارد؟
مکان	مدل بدون کد مکانی چه چیزی را نمی‌داند و embedding مطلق، bias نسبی، RoPE دوبعدی و درونیابی چه تفاوتی دارند؟
معماری‌ها	ViT، DeiT، Swin، PVT، MViT، MaxViT و Hierarchical چگونه مقیاس و وضوح را مدیریت می‌کنند؟
کارایی	توجه پنجره‌ای، پراکنده، خطی، FlashAttention، حذف/ادغام توکن و بسته‌بندی دنباله چه مبادله‌ای دارند؟
کنتراستی	InfoNCE، دما، batch منفی‌ها، head projection و augmentation چگونه هندسه نمایش را تعیین می‌کنند؟
خودتقطیر	DINO، BYOL و خانواده‌های جدید چگونه بدون منفی صریح از پاسخ ثابت اجتناب می‌کنند؟
کاهش افزونگی	Barlow Twins و VICReg چگونه واریانس و کوواریانس را کنترل می‌کنند؟
مدل سازی ماسک شده	iBOT، MAE، BEiT و data2vec در فضای هدف و معماری encoder-decoder چه تفاوتی دارند؟
پیش‌بینی نهفته	I-JEPA چرا به جای پیکسل، بلوک هدف را در فضای نمایش پیش‌بینی می‌کند؟
مدل بنیادی	داده‌گزینی، مقیاس، ویژگی متراکم، وضوح بالا و هم‌ترازی متن در DINOv2/v3، EVA-02 و SigLIP 2 چه نقشی دارند؟
کتابخانه‌ها	PyTorch SDPA، timm، Transformers، Lightly، MMSelfSup و مخازن رسمی چگونه در یک خط لوله قرار می‌گیرند؟
ارزیابی	linear، kNN probe، fine-tuning، ارزیابی، dense، transfer، عدم قطعیت، OOD و هزینه محاسباتی چگونه گزارش می‌شوند؟

فهرست مطالب

آ	یادداشت روش شناختی فصل یازدهم
پ	نقشه فشرده فصل
۱	۱۱ ترنسفورمرهای بینایی و یادگیری خودنظارتی
۱	۱۰۱۱ از پیکسل تا توکن: تغییر واحد محاسبه
۲	۱۰۱۱۱ وصله سازی یک عملگر خطی است
۴	۲۰۱۱۱ قانون مقیاس هزینه با اندازه وصله
۵	۳۰۱۱۱ اندازه وصله به عنوان پیشین فرکانسی
۵	۴۰۱۱۱ توکن های هم پوشان و stem کانولوشنی
۶	۵۰۱۱۱ توکن سازی سلسله مراتبی
۶	۶۰۱۱۱ توکن سازی تطبیقی: محتوا تعیین می کند چه چیزی دیده شود
۷	۷۰۱۱۱ وضوح بومی و نسبت تصویر
۸	۸۰۱۱۱ توکن شیء، ناحیه، و کد بصری گسسته
۹	۹۰۱۱۱ توکن سازی در داده های غیر RGB
۹	۲۰۱۱ جبر توجه خودکار
۹	۱۰۲۰۱۱ پرس وجو، کلید و مقدار
۱۰	۲۰۲۰۱۱ چرا تقسیم بر $\sqrt{d_k}$ ؟
۱۰	۳۰۲۰۱۱ توجه به عنوان رگرسیون هسته ای وابسته به داده
۱۱	۴۰۲۰۱۱ گرادیان نرمینه
۱۱	۵۰۲۰۱۱ توجه چندسری
۱۲	۶۰۲۰۱۱ بلوک ترنسفورمر پیش هنجار شده
۱۵	۷۰۲۰۱۱ آیا نقشه attention توضیح مدل است؟
۱۵	۳۰۱۱ اطلاعات مکانی: مدل باید بداند توکن کجاست
۱۵	۱۰۳۰۱۱ embedding مطلق آموختنی

۲۰۳۰۱۱	کد سینوسی دوبعدی	۱۵
۳۰۳۰۱۱	bias مکانی نسبی	۱۶
۴۰۳۰۱۱	RoPE دوبعدی	۱۶
۴۰۱۱	Transformer Vision از ابتدا تا یک backbone عمومی	۱۷
۱۰۴۰۱۱	معماری پایه ViT	۱۷
۲۰۴۰۱۱	توکن CLS یا pooling؟	۱۷
۳۰۴۰۱۱	ViT و نیاز به داده	۱۸
۴۰۴۰۱۱	DeiT: داده کارآمدی و توکن تقطیر	۱۸
۵۰۴۰۱۱	هیبرید CNN-Transformer	۱۸
۵۰۱۱	معماری‌های سلسله‌مراتبی و attention محلی	۱۸
۱۰۵۰۱۱	Transformer Vision Pyramid	۱۸
۲۰۵۰۱۱	Swin و پنجره جابه‌جاشونده	۱۹
۳۰۵۰۱۱	MViT، attention pooling و Hierarchical	۱۹
۴۰۵۰۱۱	MaxViT و attention چندمحوری	۱۹
۵۰۵۰۱۱	میدان تعامل و فاصله attention	۲۰
۶۰۱۱	کارایی: attention، FLOPs حافظه و حرکت داده	۲۰
۱۰۶۰۱۱	مسئله درجه دو	۲۰
۲۰۶۰۱۱	خانواده‌های کاهش هزینه	۲۱
۳۰۶۰۱۱	FlashAttention: تغییر الگوریتم، نه تقریب تابع	۲۱
۴۰۶۰۱۱	کاهش توکن و خطر سوگیری	۲۳
۷۰۱۱	یادگیری خودنظارتی: مسئله دقیق چیست؟	۲۳
۱۰۷۰۱۱	دو نما و گروه تبدیل	۲۳
۲۰۷۰۱۱	سه فضای متفاوت	۲۴
۸۰۱۱	یادگیری کنتراستی و InfoNCE	۲۴
۱۰۸۰۱۱	تابع هدف NT-Xent	۲۴
۲۰۸۰۱۱	alignment و uniformity	۲۶
۳۰۸۰۱۱	ارتباط با کران اطلاعات متقابل و احتیاط لازم	۲۶
۴۰۸۰۱۱	SimCLR	۲۷
۵۰۸۰۱۱	MoCo: dictionary پویا و encoder تکرانه‌ای	۲۹
۶۰۸۰۱۱	negative false و داده با ساختار گروهی	۲۹
۹۰۱۱	خوشه‌بندی آنلاین و SwAV	۲۹
۱۰۰۱۱	اروش‌های بدون negative صریح: DINO و SimSiam BYOL	۳۰

۳۰	BYOL۱.۱۰.۱۱ و نقش عدم تقارن
۳۰	SimSiam۲.۱۰.۱۱
۳۰	DINO۳.۱۰.۱۱ خود تقطیر با centering و sharpening
۳۳	۴.۱۰.۱۱ چرا ویژگی‌های متراکم در DINO مهم شدند؟
۳۳	۱۱.۱۱ کاهش افزونگی: Twins Barlow و VICReg
۳۳	Twins Barlow۱.۱۱.۱۱
۳۳	VICReg۳.۱۱.۱۱
۳۴	۲.۱۱ فروپاشی نمایش و ابزارهای تشخیص
۳۴	۱.۱۲.۱۱ فروپاشی کامل و فروپاشی بعدی
۳۵	۲.۱۲.۱۱ چک لیست پایش در آموزش
۳۶	۳.۱۱ مدل سازی ماسک شده تصویر
۳۶	۱.۱۳.۱۱ صورت بندی عمومی
۳۶	BEiT۲.۱۳.۱۱ و هدف گسسته
۳۶	MAE۳.۱۳.۱۱ و بازسازی پیکسل
۳۸	SimMIM۴.۱۳.۱۱
۳۹	۵.۱۳.۱۱ tokenizer iBOT: آنلین
۳۹	۶.۱۳.۱۱ vec:۲data هدف latent contextualized
۳۹	۷.۱۳.۱۱ mask شکل
۴۰	۱۴.۱۱ I-JEPA و پیش بینی در فضای نهفته
۴۰	۱.۱۴.۱۱ چرا sampling بلوک مهم است؟
۴۰	۲.۱۴.۱۱ JEPA در برابر MAE
۴۱	۵.۱۱ مدل های بنیادی خود نظارتی
۴۱	۱.۱۵.۱۱ DINO۲: روش، داده و مهندسی مقیاس
۴۱	۲.۱۵.۱۱ token ها register
۴۱	۳.۱۵.۱۱ DINO۳ و مقیاس جدید
۴۲	۴.۱۵.۱۱ EVA۲
۴۲	۱۶.۱۱ پیش آموزش تصویر- متن: SigLIP CLIP و SigLIP ۲
۴۲	۱.۱۶.۱۱ SigLIP
۴۲	۲.۱۶.۱۱ SigLIP ۲
۴۳	۷.۱۱ انتقال نمایش به وظایف پایین دست
۴۳	۱.۱۷.۱۱ kNN و probe linear
۴۳	۳.۱۷.۱۱ fine-tuning کامل و لایه ای

۴۳	tuning prompt و LoRA adapter	۳۰۱۷۰۱۱
۴۴	کم‌پرچسب	۴۰۱۷۰۱۱
۴۴	انتقال متراکم: از token patch تا نقشه ویژگی	۱۸۰۱۱
۴۵	correspondence و تطبیق با ویژگی خودنظارتی	۱۰۱۸۰۱۱
۴۵	ارزیابی یک نمایش: یک عدد کافی نیست	۹۰۱۱
۴۵	اپروتکل‌های اصلی	۱۰۱۹۰۱۱
۴۶	پروتکل probe linear عادلانه	۱۰۱۹۰۱۱
۴۶	ارزیابی بدون نشت	۳۰۱۹۰۱۱
۴۶	عدم قطعیت و OOD	۴۰۱۹۰۱۱
۴۷	کتابخانه‌ها: از kernel تا recipe خودنظارتی	۲۰۰۱۱
۴۷	PyTorch	۱۰۲۰۰۱۱
۴۷	timm و torchvision	۲۰۲۰۰۱۱
۴۸	Transformers Face Hugging	۳۰۲۰۰۱۱
۴۸	MMSelfSup و solo-learn Lightly	۴۰۲۰۰۱۱
۴۸	WebDataset و داده وب مقیاس	۵۰۲۰۰۱۱
۴۸	مهندسی آموزش در مقیاس	۲۰۱۱
۴۹	batch مؤثر و accumulation gradient	۱۰۲۱۰۱۱
۴۹	EMA در آموزش توزیع شده	۲۰۲۱۰۱۱
۴۹	precision mixed	۳۰۲۱۰۱۱
۴۹	decay weight و پارامترهای مستثنا	۴۰۲۱۰۱۱
۴۹	schedule متداول	۵۰۲۱۰۱۱
۵۰	checkpoints و بازتولیدپذیری	۶۰۲۱۰۱۱
۵۰	طراحی آزمایش‌های دقیق	۲۰۲۱۱
۵۰	آزمایش ۱: اندازه وصله	۱۰۲۲۰۱۱
۵۰	آزمایش ۲: کدگذاری مکان	۲۰۲۲۰۱۱
۵۱	آزمایش ۳: دمای InfoNCE	۳۰۲۲۰۱۱
۵۱	آزمایش ۴: نسبت mask	۴۰۲۲۰۱۱
۵۱	آزمایش ۵: فضای هدف	۵۰۲۲۰۱۱
۵۱	آزمایش ۶: augmentation دامنه‌ای	۶۰۲۲۰۱۱
۵۱	آزمایش ۷: پیش‌آموزش درون دامنه و برون دامنه	۷۰۲۲۰۱۱
۵۲	آزمایش ۸: pruning/merging token	۸۰۲۲۰۱۱

۵۲	۹۰۲۲۰۱۱. آزمون‌های features dense frozen
۵۲	۹۰۲۲۰۱۱. آزمون‌های ۱۰: ممیزی فروپاشی
۵۳	۲۳۰۱۱. عیب‌یابی سیستماتیک
۵۴	۲۴۰۱۱. مرزهای پژوهش در ۲۰۲۵-۲۰۲۶
۵۴	۹۰۲۴۰۱۱. ویژگی متراکم در مقیاس بنیادین
۵۴	۹۰۲۴۰۱۱. ترکیب چند هدف
۵۴	۹۰۲۴۰۱۱. وضوح و نسبت تصویر بومی
۵۴	۹۰۲۴۰۱۱. توکن پویا و بودجه تطبیقی
۵۴	۹۰۲۴۰۱۱. خودنظارتی بدون augmentation دست‌ساز شدید
۵۵	۹۰۲۴۰۱۱. هم‌ترازی متن و حفظ هندسه
۵۵	۹۰۲۴۰۱۱. داده‌گزینی و حاکمیت داده
۵۵	۹۰۲۴۰۱۱. ویژگی‌ها به‌عنوان زیرساخت
۵۵	۲۵۰۱۱. الگوریتم‌های مرجع فصل
۵۷	۹۰۲۶۰۱۱. کارگاه یکپارچه: از صفر تا مدل آماده
۵۷	۹۰۲۶۰۱۱. مسیر A: فهم از صفر
۵۸	۹۰۲۶۰۱۱. مسیر B: InfoNCE و collapse
۵۸	۹۰۲۶۰۱۱. مسیر C: MAE
۵۸	۹۰۲۶۰۱۱. مسیر D: مدل آماده
۵۹	۷۰۱۱. تمرین‌های نظری و اثباتی
۶۰	۲۸۰۱۱. تمرین‌های کدنویسی
۶۱	۹۰۱۱. پروژه‌های پایان فصل
۶۱	۹۰۲۹۰۱۱. پروژه ۱: آزمایشگاه توکن‌سازی
۶۱	۹۰۲۹۰۱۱. پروژه ۲: بازسازی پنج خانواده SSL
۶۱	۹۰۲۹۰۱۱. پروژه ۳: خودنظارتی دامنه تخصصی
۶۲	۹۰۲۹۰۱۱. پروژه ۴: dense correspondence با frozen backbone
۶۲	۹۰۲۹۰۱۱. پروژه ۵: مدل بودجه تطبیقی
۶۲	۹۰۲۹۰۱۱. پروژه ۶: ممیزی مدل بنیادی
۶۲	۹۰۲۹۰۱۱. پروژه ۷: روش جدید ضد فروپاشی
۶۲	۹۰۲۹۰۱۱. پروژه ۸: کتابخانه آموزشی فارسی
۶۲	۳۰۰۱۱. پیرامیت پیشنهادی برای Codex
۶۴	۳۱۰۱۱. تقسیم پیشنهادی ویدئوهای ۱۵ دقیقه‌ای

۷۳	آ راهنمای ساخت، اجرا و بازتولید
۷۳	۱.آ ساخت PDF
۷۳	۲.آ ساخت محیط پایتون
۷۳	۳.آ تولید شکل ها
۷۳	۴.آ اجرای test smoke
۷۴	۵.آ نمونه استفاده از مدل کتابخانه ای
۷۴	۶.آ ساخت manifest و checksum
۷۴	۷.آ پیش پرواز علمی

فصل ۱۱

ترنسفورمرهای بینایی و یادگیری خودنظارتی

اهداف یادگیری

در پایان فصل، دانشجو باید بتواند:

- عملگر patchify/unpatchify را با نمادگذاری تانسوری و کد پیاده‌سازی کند؛
- تعداد توکن، پارامتر و هزینه توجه را برای وضوح و اندازه وصله دلخواه محاسبه کند؛
- توجه مقیاس‌شده و چندسری را از روی ضرب داخلی و نرمینه استخراج کند؛
- تفاوت embedding مکان مطلق، bias نسبی و RoPE را تحلیل کند؛
- یک ViT پیش‌هنجارشده را از صفر بسازد و گرادین آن را بررسی کند؛
- InfoNCE و مشتق آن نسبت به logits و دما را توضیح دهد؛
- سازوکار ضد فروپاشی در MoCo، BYOL، DINO، Barlow Twins، VICReg، MAE و I-JEPA را مقایسه کند؛
- معیارهای تشخیص فروپاشی کامل و بعدی را محاسبه کند؛
- یک backbone پیش‌آموخته را برای linear، probe، استخراج ویژگی یا fine-tuning به‌کار گیرد؛
- یک آزمایش خودنظارتی بازتولیدپذیر با ثبت داده، augmentation، seed، schedule هزینه و معیار بسازد.

۱۰.۱۱ از پیکسل تا توکن: تغییر واحد محاسبه

در شبکه کانولوشنی، واحد پایه محاسبه معمولاً یک مکان روی map feature است. در ViT، تصویر ابتدا به واحدهایی تبدیل می‌شود که در ادبیات «توکن» نام دارند. توکن لزوماً یک وصله ثابت نیست؛ می‌تواند

حاصل یک کانولوشن هم‌پوشان، یک ناحیه شیء، یک خوشه از نقاط، یک لوله زمانی در ویدئو یا یک بردار آموخته شده باشد. بنابراین پیش از هر بحث درباره attention باید پرسید:

مدل دقیقاً چه چیزهایی را به عنوان عناصر دنباله می بیند و چه اطلاعاتی پیش از اولین بلوک ترنسفورمر حذف شده است؟

فرض کنید تصویر

$$\mathbf{X} \in \mathbb{R}^{H \times W \times C} \quad (1.11)$$

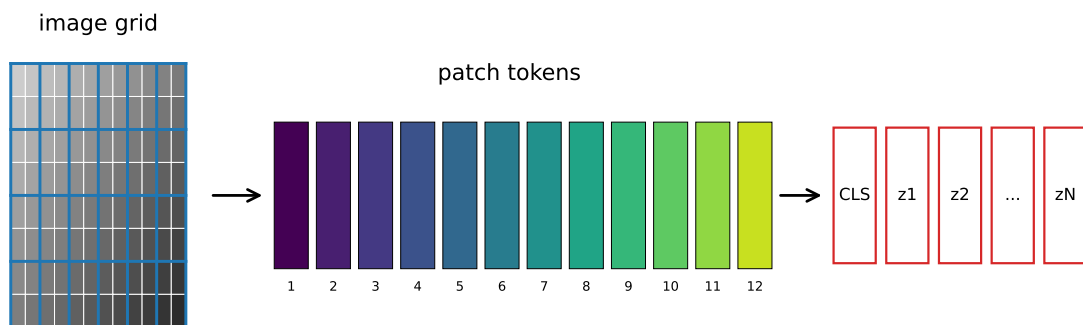
با وصله مربعی $P \times P$ و بدون هم‌پوشانی تقسیم شود. اگر $P \mid H$ و $P \mid W$ ، تعداد وصله ها برابر است با

$$N = \frac{H}{P} \frac{W}{P} = \frac{HW}{P^2}. \quad (2.11)$$

هر وصله تحت می شود و با ماتریس آموختنی $\mathbf{E} \in \mathbb{R}^{(P^2 C) \times D}$ به فضای embedding می رود:

$$\mathbf{z}_i = \text{vec}(\mathbf{X}_i) \mathbf{E} + \mathbf{b}, \quad i = 1, \dots, N. \quad (3.11)$$

اگر توکن طبقه بندی افزوده شود، طول دنباله $N + 1$ خواهد بود.



شکل ۱۰۱۱: تبدیل شبکه پیکسلی به دنباله توکن ها. ترتیب خطی باید با هندسه دوبعدی و embedding مکانی سازگار باقی بماند.

۱۰۱۰۱۱ وصله سازی یک عملگر خطی است

عمل patchify را می توان با یک ماتریس جایگشتی و انتخابی نوشت. اگر تصویر برداری شده را $\mathbf{x} \in \mathbb{R}^{HWC}$ بنامیم، آنگاه

$$\mathbf{P}_{\text{patch}} \mathbf{x} = \begin{bmatrix} \text{vec}(\mathbf{X}_1) \\ \vdots \\ \text{vec}(\mathbf{X}_N) \end{bmatrix} \in \mathbb{R}^{N \times P^2 C}. \quad (4.11)$$

پس embedding وصله‌ها یک نگاشت خطی مرکب است:

$$Z = (P_{\text{patch}}x)E + \mathbf{1}b^T. \quad (5.11)$$

این مشاهده دو پیام دارد. نخست آنکه مرحله patchify ذاتاً یادگیری ندارد و تنها بازآرایی است؛ دوم آنکه projection وصله‌ها دقیقاً با یک کانولوشن با kernel و stride برابر P هم‌ارز است. در قرارداد NCHW:

$$\text{ConvD}(C, D, k = P, s = P) \iff \text{patchify} + \text{projection linear}. \quad (6.11)$$

این هم‌ارزی برای انتقال وزن، export مدل و فهم تفاوت پیاده‌سازی‌ها مهم است.

کارگاه کد و آزمایش: وصله‌سازی و بازسازی دقیق

فایل code/patchify.py عملگر رفت و برگشت را پیاده‌سازی می‌کند. آزمون اصلی باید خطای بازسازی صفر تولید کند؛ اگر ترتیب محورها در permute اشتباه باشد، تصویر ظاهراً ابعاد صحیح دارد اما پیکسل‌ها درون وصله‌ها جابه‌جا می‌شوند.

```

1 from __future__ import annotations
2 import torch
3
4
5 def patchify(x: torch.Tensor, patch: int) -> torch.Tensor:
6     """B,C,H,W -> B,N,(C*patch*patch). H,W must be divisible by patch.
7     """
8     if x.ndim != 4:
9         raise ValueError('x must have shape [B,C,H,W]')
10    b, c, h, w = x.shape
11    if h % patch or w % patch:
12        raise ValueError('height and width must be divisible by patch
13        size')
14    y = x.reshape(b, c, h // patch, patch, w // patch, patch)
15    y = y.permute(0, 2, 4, 1, 3, 5).contiguous()
16    return y.reshape(b, (h // patch) * (w // patch), c * patch * patch
17    )
18
19 def unpatchify(tokens: torch.Tensor, patch: int, height: int, width:
20 int, channels: int) -> torch.Tensor:
21     """Inverse of patchify for a known image shape."""
22    b, n, d = tokens.shape
23    gh, gw = height // patch, width // patch
24    if n != gh * gw or d != channels * patch * patch:
25        raise ValueError('token shape is inconsistent with requested
26        image shape')
27    y = tokens.reshape(b, gh, gw, channels, patch, patch)

```

```

24 y = y.permute(0, 3, 1, 4, 2, 5).contiguous()
25 return y.reshape(b, channels, height, width)
26
27
28 if __name__ == '__main__':
29     torch.manual_seed(0)
30     x = torch.randn(2, 3, 32, 48)
31     t = patchify(x, 8)
32     xr = unpatchify(t, 8, 32, 48, 3)
33     err = (x - xr).abs().max().item()
34     print({'tokens': tuple(t.shape), 'max_reconstruction_error': err})
35     assert err == 0.0

```

۲۰۱۱ قانون مقیاس هزینه با اندازه وصله

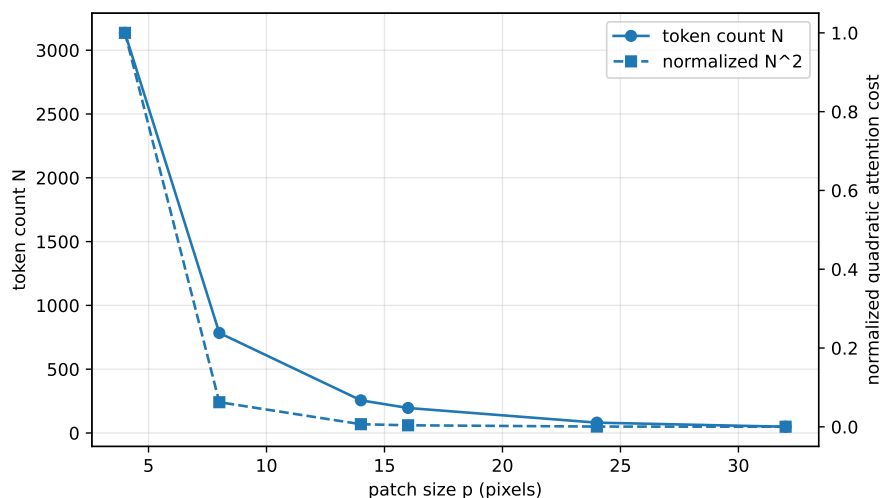
هزینه attention سراسری شامل تشکیل QK^T است. با حذف ثابت‌ها:

$$\mathcal{C}_{\text{attn}} = \mathcal{O}(N^2 D), \quad \mathcal{M}_{\text{attn}} = \mathcal{O}(N^2). \quad (۷۰۱۱)$$

با جایگذاری رابطه (۲۰۱۱):

$$\mathcal{C}_{\text{attn}} = \mathcal{O}\left(\frac{H^2 W^2 D}{P^2}\right). \quad (۸۰۱۱)$$

بنابراین نصف کردن اندازه وصله، در وضوح ثابت، تعداد توکن را تقریباً چهار برابر و ماتریس attention را حدود شانزده برابر می‌کند. این رابطه توضیح می‌دهد چرا تغییر ظاهراً کوچک $P = ۱۶$ به $P = ۸$ می‌تواند حافظه را به‌طور چشمگیر افزایش دهد.



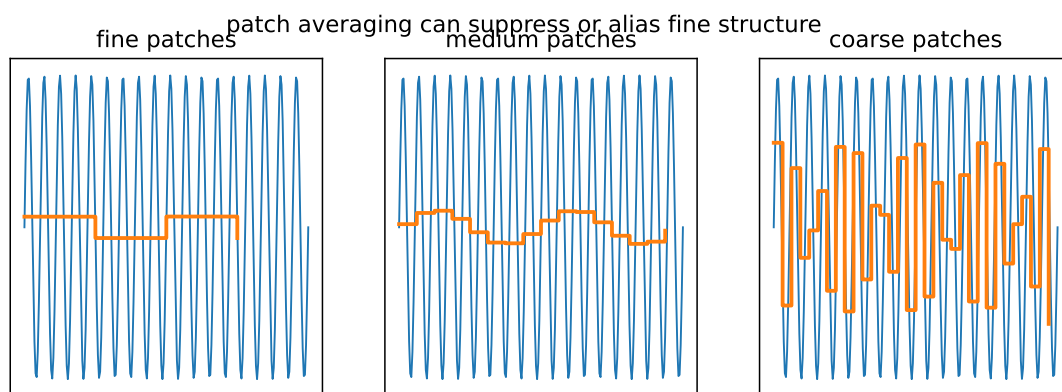
شکل ۲۰۱۱: اندازه وصله نه تنها وضوح نمایش، بلکه هزینه درجه‌دو attention را تعیین می‌کند.

قانون مهندسی

در گزارش مدل، نوشتن نام ViT-B کافی نیست. باید حداقل وضوح ورودی، اندازه وصله، تعداد توکن، precision batch و نوع attention ذکر شود؛ زیرا یک backbone یکسان در وضوح‌های متفاوت هزینه و حتی رفتار بازنمایی متفاوتی دارد.

۳.۱۰.۱۱ اندازه وصله به عنوان پیشین فرکانسی

وصله بزرگ، پیش از هر تعامل بین وصله‌ای، جزئیات را در یک بردار فشرده می‌کند. اگر projection اولیه نتواند مؤلفه‌های پرفرکانس را حفظ کند، اطلاعات از دست رفته در لایه‌های بعد باز نمی‌گردد. از دید نمونه‌برداری، شبکه‌ای از توکن‌ها با گام P ساخته می‌شود؛ بدون stem ضدآلیاس، الگوهای ریز ممکن است به ساختار کاذب در شبکه توکن تبدیل شوند.



شکل ۳.۱۱: نمای آموزشی اثر وصله درشت بر ساختار ریز. projection آموختنی می‌تواند بهتر از میانگین ساده باشد، اما محدودیت نمونه‌برداری حذف نمی‌شود.

۴.۱۰.۱۱ توکن‌های هم‌پوشان و stem کانولوشنی

اگر kernel بزرگ‌تر از stride باشد، وصله‌ها هم‌پوشان می‌شوند. برای kernel K ، stride S و padding A ، تعداد توکن در هر محور

$$N_H = \left\lfloor \frac{H + 2A - K}{S} \right\rfloor + 1, \quad N_W = \left\lfloor \frac{W + 2A - K}{S} \right\rfloor + 1. \quad (9.11)$$

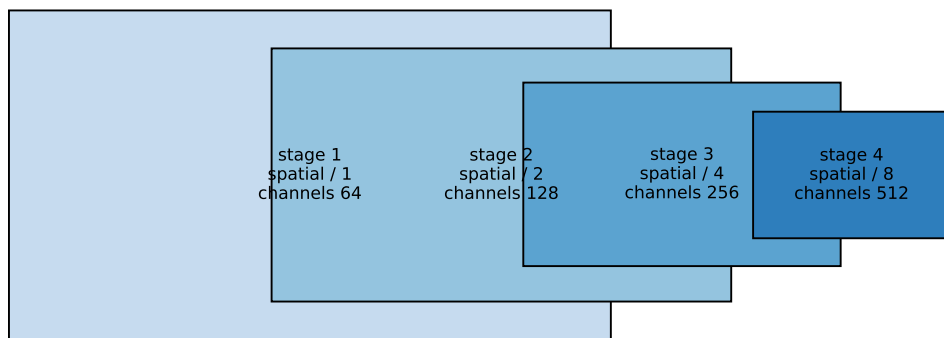
هم‌پوشانی پیوستگی مرز وصله‌ها را بهتر می‌کند، اما محاسبه و افزودن را افزایش می‌دهد. stem چندلایه کانولوشنی می‌تواند پیش از transformer لبه و بافت محلی را استخراج و سپس دنباله کوتاه‌تری بسازد. این طراحی در معماری‌های هیبریدی و برخی ViT‌های داده‌کارآمد رایج است.

۵.۱.۱۱ توکن سازی سلسله مراتبی

در های prediction، dense backbone نگه داشتن وضوح ثابت در همه لایه ها پرهزینه است. معماری سلسله مراتبی پس از چند بلوک، توکن های مجاور را ادغام می کند و تعداد کانال را افزایش می دهد:

$$(H_l, W_l, D_l) \rightarrow \left(\frac{H_l}{2}, \frac{W_l}{2}, 2D_l \right). \quad (10.11)$$

این ساختار، هرم ویژگی مشابه CNN ایجاد می کند و اتصال به های decoder تشخیص و قطعه بندی را آسان می سازد.



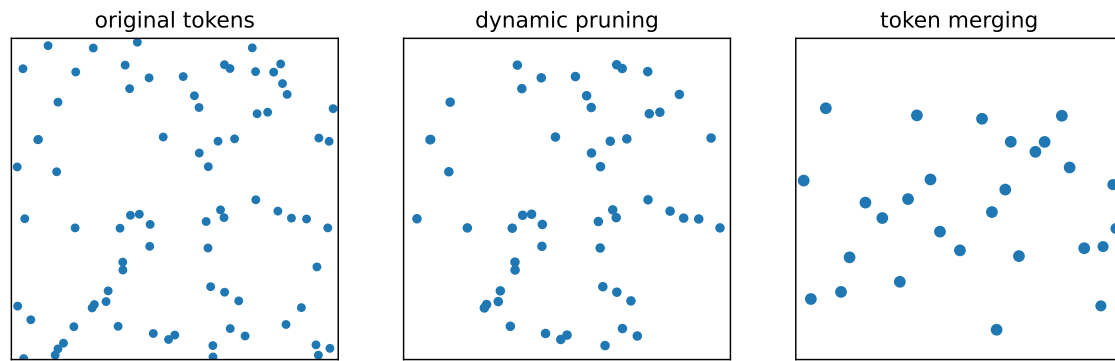
شکل ۴.۱۱: نمایش سلسله مراتبی با کاهش وضوح و افزایش کانال. طراحی دقیق نسبت ها به معماری وابسته است.

۶.۱.۱۱ توکن سازی تطبیقی: محتوا تعیین می کند چه چیزی دیده شود

وصله ثابت مستقل از محتواست. در مقابل، TokenLearner نقشه های توجه فضایی می آموزد و چند توکن خلاصه محتوا محور تولید می کند. DynamicViT در میانه شبکه، توکن های کم اهمیت را حذف می کند و ToMe توکن های مشابه را ادغام می کند. اگر N_l تعداد توکن در لایه l باشد، هزینه کل تقریبی

$$\sum_{l=1}^L \mathcal{O}(N_l^2 D) \quad (11.11)$$

است؛ بنابراین کاهش زودهنگام N_l سود بیشتری دارد. با این حال، حذف توکن ممکن است ساختار کوچک یا اقلیت را نابود کند و ادغام توکن ممکن است مرزهای اشیاء نزدیک را مخلوط سازد. معیار کارایی باید علاوه بر دقت طبقه بندی، وظایف مترکم و گروه های دشوار را نیز بررسی کند.



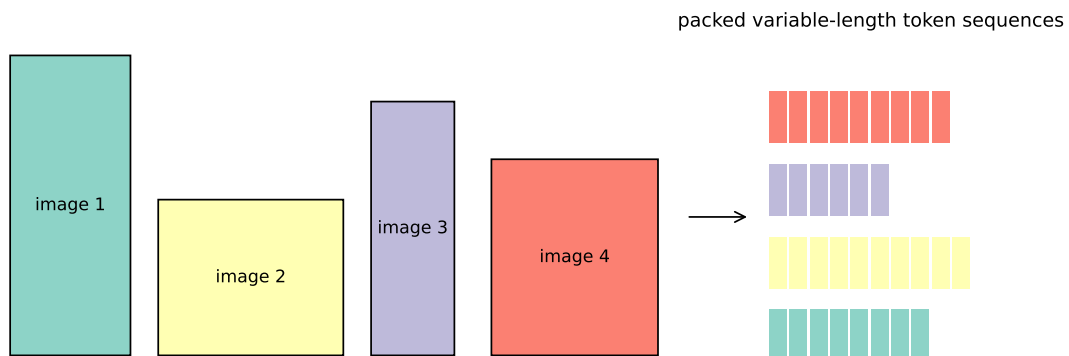
شکل ۵.۱۱: سه راهبرد: حفظ همه توکن‌ها، حذف محتوای محور و ادغام توکن‌های مشابه. کاهش FLOPs معادل حفظ اطلاعات نیست.

۷.۱.۱۱ وضوح بومی و نسبت تصویر

تغییر اندازه همه تصاویر به مربع ثابت، هندسه و مقیاس اشیاء را تغییر می‌دهد. NaViT تصاویر با نسبت و وضوح متفاوت را به دنباله‌های با طول متغیر تبدیل و چند دنباله را در یک بسته محاسباتی قرار می‌دهد. برای جلوگیری از توجه میان دو تصویر متفاوت، mask بلوکی لازم است:

$$M_{ij} = \begin{cases} 0, & s(i) = s(j), \\ -\infty, & s(i) \neq s(j), \end{cases} \quad (12.11)$$

که $s(i)$ شناسه تصویر مالک توکن i است. FlexiViT با تصادفی کردن اندازه وصله در آموزش، یک مجموعه وزن را برای چند بوجه محاسباتی آماده می‌کند.



شکل ۶.۱۱: بسته‌بندی دنباله‌های با طول متفاوت؛ mask attention باید مرز میان تصاویر را حفظ کند.

کارگاه کد و آزمایش: بسته‌بندی دنباله با طول متغیر

```
1 from __future__ import annotations
2 from dataclasses import dataclass
3 import torch
```

```

4
5
6 @dataclass
7 class PackedBatch:
8     tokens: torch.Tensor
9     segment_ids: torch.Tensor
10    lengths: list[int]
11
12
13 def pack_sequences(sequences: list[torch.Tensor]) -> PackedBatch:
14     if not sequences or any(x.ndim != 2 for x in sequences):
15         raise ValueError('provide a non-empty list of [N_i,D] tensors'
16     )
17     d = sequences[0].shape[1]
18     if any(x.shape[1] != d for x in sequences):
19         raise ValueError('embedding dimensions must match')
20     tokens = torch.cat(sequences, dim=0)
21     ids = torch.cat([torch.full((x.shape[0],), i, dtype=torch.long,
22 device=x.device)
23                     for i, x in enumerate(sequences)])
24     return PackedBatch(tokens, ids, [x.shape[0] for x in sequences])
25
26 def block_diagonal_mask(segment_ids: torch.Tensor) -> torch.Tensor:
27     """True means attention is permitted within the same packed image.
28     """
29     return segment_ids[:, None] == segment_ids[None, :]
30
31 if __name__ == '__main__':
32     torch.manual_seed(6)
33     seqs=[torch.randn(n,32) for n in [12,7,20,5]]
34     packed=pack_sequences(seqs); mask=block_diagonal_mask(packed.
35 segment_ids)
36     print({'packed':tuple(packed.tokens.shape), 'lengths':packed.
37 lengths, 'allowed_fraction':float(mask.float().mean())})
38     assert mask.shape[0] == sum(packed.lengths)

```

در سامانه واقعی، استفاده از NestedTensor، packing sequence یا kernel اختصاصی باید با profiler سنجیده شود؛ padding ساده گاهی با وجود محاسبه اضافه، روی GPU سریع تر از ساختار بسیار نامنظم است.

۸.۱.۱۱ توکن شیء، ناحیه، و کد بصری گسسته

سه مفهوم نباید با هم مخلوط شوند:

۱. توکن وصله: یک مکان شبکه منظم را نمایندگی می کند؛

۲. توکن ناحیه/شیء: از pooling یا query آموختنی برای یک موجودیت احتمالی ساخته می شود؛

۳. توکن گسسته بصری: شاخص یک codebook مانند خروجی tokenizer در BEiT است.

توکن گسسته برای هدف طبقه بندی ماسک شده مناسب است، اما خطای tokenizer و ظرفیت codebook به هدف پیش آموزش منتقل می شود. توکن شیء برای شمارش و استدلال موجودیت محور مفید است، اما معمولاً نیازمند سازوکار assignment یا query-based decoder است.

۹.۱.۱۱ توکن سازی در داده های غیر RGB

در تصاویر چندطیفی، بعد کانال ممکن است ده ها یا صدها باشد. سه انتخاب رایج وجود دارد: projection مشترک همه کانال ها، توکن جدا برای هر کانال، یا ترکیب توکن مکانی و embedding کانال. در حجم سه بعدی، وصله $P_x \times P_y \times P_z$ و در ویدئو tubelet با اندازه $P_t \times P_h \times P_w$ است. تعداد توکن و هزینه attention اکنون با حاصل ضرب زمان/عمق نیز رشد می کند و توجه factorized یا سلسله مراتبی ضروری می شود.

خطای رایج

توکن کوچک همیشه بهتر نیست. توکن کوچک، جزئیات را حفظ می کند اما هزینه را افزایش می دهد، احتمال overfitting به بافت را بالا می برد و ممکن است به batch کوچک تر و تخمین آماری ضعیف تر منجر شود. اندازه وصله یک ابرپارامتر «نمایش - محاسبه - داده» است، نه صرفاً تنظیم هندسی.

۲.۱۱ جبر توجه خودکار

۱.۲.۱۱ پرس وجو، کلید و مقدار

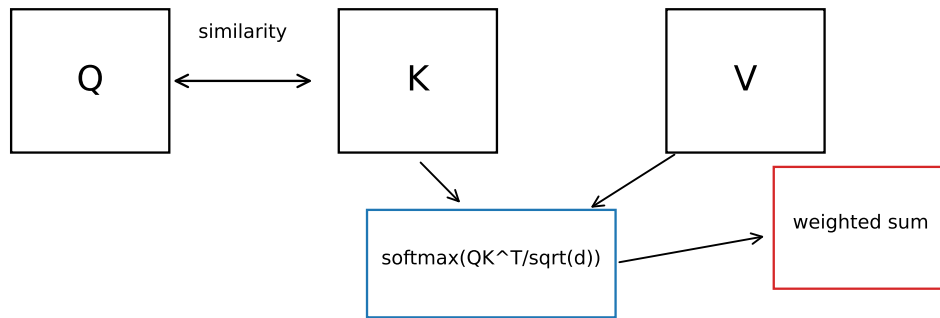
برای ورودی $X \in \mathbb{R}^{N \times D}$ ، سه projection خطی ساخته می شود:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad (13.11)$$

که معمولاً $Q, K \in \mathbb{R}^{N \times d_k}$ و $V \in \mathbb{R}^{N \times d_v}$. خروجی توجه مقیاس شده برابر است با

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V. \quad (14.11)$$

نرمینه روی بعد کلیدها اعمال می شود؛ در نتیجه هر سطر ماتریس وزن جمع یک دارد. M می تواند mask padding، پنجره ای یا مرز بسته بندی باشد.



شکل ۷.۱۱: توجه به عنوان شباهت query-key و میانگین وزن‌دار value.

۲۰.۲۰۱۱ چرا تقسیم بر $\sqrt{d_k}$ ؟

فرض کنید مؤلفه‌های query و key مستقل، با میانگین صفر و واریانس یک باشند. برای logit

$$s_{ij} = \mathbf{q}_i^T \mathbf{k}_j = \sum_{r=1}^{d_k} q_{ir} k_{jr}, \quad (15.11)$$

داریم

$$\mathbb{E}[s_{ij}] = 0, \quad \text{Var}(s_{ij}) = d_k. \quad (16.11)$$

با رشد d_k ، logits بزرگ و softmax اشباع می‌شود. مقیاس $1/\sqrt{d_k}$ واریانس را تقریباً یک نگه می‌دارد و گرادینان را در ناحیه قابل استفاده حفظ می‌کند. این استدلال دقیقاً اثبات بهینگی نیست؛ وابستگی مؤلفه‌ها، normalization و وزن‌های آموخته‌شده فرض استقلال را نقض می‌کنند، اما مقیاس از رشد سیستماتیک جلوگیری می‌کند.

۳۰.۲۰۱۱ توجه به عنوان رگرسیون هسته‌ای وابسته به داده

برای query ثابت i ، وزن

$$a_{ij} = \frac{\exp(\mathbf{q}_i^T \mathbf{k}_j / \sqrt{d_k})}{\sum_m \exp(\mathbf{q}_i^T \mathbf{k}_m / \sqrt{d_k})} \quad (17.11)$$

یک kernel نرمال‌شده است و خروجی

$$\mathbf{y}_i = \sum_j a_{ij} \mathbf{v}_j \quad (18.11)$$

مشابه رگرسیون Nadaraya-Watson در فضای ویژگی است، با این تفاوت که metric و مقادیر هم‌زمان آموخته می‌شوند. همچنین می‌توان attention را پیام‌رسانی روی یک گراف کامل داده‌محور دانست که وزن یال‌ها در هر نمونه تغییر می‌کند.

۴.۲.۱۱ گرادیان نرمینه

برای $a = \text{softmax}(s)$ ، ژاکوبین

$$\frac{\partial a_i}{\partial s_j} = a_i(\delta_{ij} - a_j) \quad (19.11)$$

یا به شکل ماتریسی

$$\mathbf{J}_{\text{softmax}} = \text{diag}(\mathbf{a}) - \mathbf{a}\mathbf{a}^T \quad (20.11)$$

است. اگر توزیع بسیار تیز شود، بیشتر a_i ها نزدیک صفر و یک مقدار نزدیک یک می‌شود؛ در این حالت بسیاری از عناصر ژاکوبین کوچک‌اند. از این رو دما، مقیاس logits و normalization نه فقط خروجی بلکه شرطی بودن بهینه‌سازی را تغییر می‌دهند.

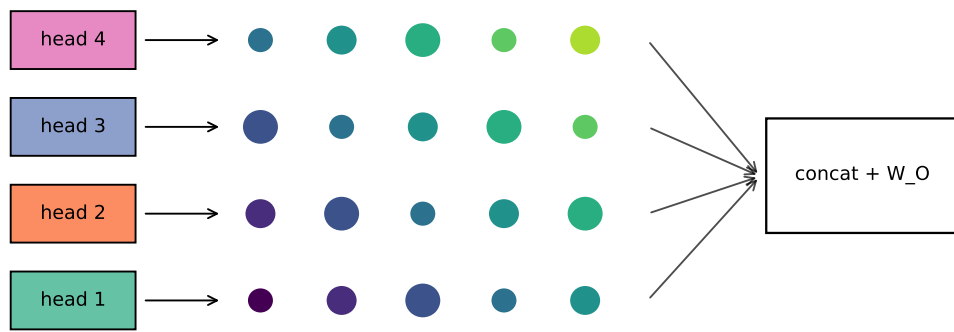
۵.۲.۱۱ توجه چندسری

در توجه چندسری، فضای embedding به h زیرفضا تقسیم می‌شود:

$$\text{MHA}(\mathbf{X}) = \text{Concat}(\mathbf{O}_1, \dots, \mathbf{O}_h) \mathbf{W}_O, \quad (21.11)$$

$$\mathbf{O}_r = \text{Attn}(\mathbf{X}\mathbf{W}_Q^{(r)}, \mathbf{X}\mathbf{W}_K^{(r)}, \mathbf{X}\mathbf{W}_V^{(r)}). \quad (22.11)$$

سرهای می‌توانند روابط مکانی یا معنایی متفاوتی بیاموزند، اما هیچ تضمینی برای تخصص‌یافتگی یا عدم افزونگی وجود ندارد. pruning سرها و تحلیل شباهت نشان می‌دهد بعضی سرها قابل حذف‌اند؛ بنابراین «تعداد سر بیشتر» ظرفیت مؤثر را لزوماً به همان نسبت افزایش نمی‌دهد.



شکل ۸.۱۱: هر سر projection و ماتریس وزن مستقل دارد؛ خروجی‌ها الحاق و دوباره projection می‌شوند.

۶.۲.۱۱ بلوک ترنسفورمر پیش‌هنجارشده

یک بلوک رایج Pre-LN به صورت

$$U_l = X_l + \text{DropPath}(\text{MHA}(\text{LN}(X_l))), \quad (23.11)$$

$$X_{l+1} = U_l + \text{DropPath}(\text{MLP}(\text{LN}(U_l))) \quad (24.11)$$

است. مسیر هویت باعث می‌شود گرادیان یک مسیر مستقیم داشته باشد. در Post-LN normalization، پس از جمع residual می‌آید. Pre-LN معمولاً برای شبکه‌های عمیق پایدارتر است، هرچند های recipe جدید با scaling و initialization می‌توانند Post-LN را نیز قابل آموزش سازند.

MLP معمولاً

$$\text{MLP}(x) = W_2 \phi(W_1 x + b_1) + b_2 \quad (25.11)$$

با نسبت گسترش $r = D_{\text{hidden}}/D$ است. نسخه‌های gated مانند SwiGLU دو projection می‌سازند و حاصل ضرب دروازه‌ای را به کار می‌برند.

کارگاه کد و آزمایش: ساخت ViT آموزشی با kernel بهینه توجه

```
1 from __future__ import annotations
2 import math
3 import torch
4 from torch import nn
5 from torch.nn import functional as F
6
7
8 class PatchEmbed(nn.Module):
9     def __init__(self, image_size=32, patch_size=4, in_chans=3, dim
```

```
=192):  
    super().__init__()  
    if image_size % patch_size:  
        raise ValueError('image_size must be divisible by  
patch_size')  
    self.grid = image_size // patch_size  
    self.num_patches = self.grid ** 2  
    self.proj = nn.Conv2d(in_chans, dim, kernel_size=patch_size,  
stride=patch_size)  
  
def forward(self, x):  
    return self.proj(x).flatten(2).transpose(1, 2)  
  
class Attention(nn.Module):  
    def __init__(self, dim: int, heads: int = 6, dropout: float = 0.0)  
    :  
        super().__init__()  
        if dim % heads:  
            raise ValueError('dim must be divisible by heads')  
        self.heads = heads  
        self.head_dim = dim // heads  
        self.qkv = nn.Linear(dim, 3 * dim)  
        self.proj = nn.Linear(dim, dim)  
        self.dropout = dropout  
  
    def forward(self, x):  
        b, n, d = x.shape  
        qkv = self.qkv(x).reshape(b, n, 3, self.heads, self.head_dim)  
        q, k, v = qkv.permute(2, 0, 3, 1, 4)  
        y = F.scaled_dot_product_attention(q, k, v, dropout_p=self.  
dropout if self.training else 0.0)  
        y = y.transpose(1, 2).reshape(b, n, d)  
        return self.proj(y)  
  
class Block(nn.Module):  
    def __init__(self, dim: int, heads: int, mlp_ratio: float = 4.0,  
drop: float = 0.0):  
        super().__init__()  
        hidden = int(dim * mlp_ratio)  
        self.norm1 = nn.LayerNorm(dim)  
        self.attn = Attention(dim, heads, drop)  
        self.norm2 = nn.LayerNorm(dim)  
        self.mlp = nn.Sequential(nn.Linear(dim, hidden), nn.GELU(), nn.  
Dropout(drop),  
nn.Linear(hidden, dim), nn.Dropout(  

```

```

drop))

50
51     def forward(self, x):
52         x = x + self.attn(self.norm1(x))
53         x = x + self.mlp(self.norm2(x))
54         return x
55
56
57 class TinyViT(nn.Module):
58     def __init__(self, image_size=32, patch_size=4, num_classes=10,
59                 dim=192, depth=4, heads=6):
60         super().__init__()
61         self.patch = PatchEmbed(image_size, patch_size, 3, dim)
62         self.cls = nn.Parameter(torch.zeros(1, 1, dim))
63         self.pos = nn.Parameter(torch.zeros(1, self.patch.num_patches
64 + 1, dim))
65         self.blocks = nn.Sequential(*[Block(dim, heads) for _ in range
66 (depth)])
67         self.norm = nn.LayerNorm(dim)
68         self.head = nn.Linear(dim, num_classes)
69         nn.init.trunc_normal_(self.pos, std=0.02)
70         nn.init.trunc_normal_(self.cls, std=0.02)
71
72     def forward_features(self, x):
73         x = self.patch(x)
74         cls = self.cls.expand(x.shape[0], -1, -1)
75         x = torch.cat([cls, x], dim=1) + self.pos
76         return self.norm(self.blocks(x))
77
78     def forward(self, x):
79         return self.head(self.forward_features(x)[: , 0])
80
81
82 if __name__ == '__main__':
83     torch.manual_seed(1)
84     model = TinyViT()
85     x = torch.randn(2, 3, 32, 32)
86     y = model(x)
87     loss = y.square().mean()
88     loss.backward()
89     n_params = sum(p.numel() for p in model.parameters())
90     print({'output': tuple(y.shape), 'parameters': n_params, 'loss':
91 float(loss.detach())})
92     assert y.shape == (2, 10)
93     assert all(torch.isfinite(p.grad).all() for p in model.parameters
94 ()) if p.grad is not None)

```

این کد از `torch.nn.functional.scaled_dot_product_attention` استفاده می کند. روی سخت افزار پشتیبانی شده، PyTorch می تواند backend بهینه را انتخاب کند؛ با این حال، `mask`، `dropout dtype` و شکل تانسور تعیین می کنند `path fast` فعال شود یا نه.

۷.۲.۱۱ آیا نقشه attention توضیح مدل است؟

ماتریس attention نشان می دهد `values` چگونه در یک لایه مخلوط شده اند، اما به تنهایی اثر نهایی یک ورودی بر تصمیم را نشان نمی دهد. `residual`، `MLP` چند لایه و `projection` خروجی مسیرهای دیگری می سازند. برای تفسیر باید attention، `gradient-weighted rollout`، `perturbation attention` و آزمون علی مقایسه شوند. نقشه زیبا بدون آزمون حساسیت می تواند همراه کننده باشد.

۳.۱۱ اطلاعات مکانی: مدل باید بداند توکن بجاست

بدون embedding مکانی، self-attention نسبت به جایگشت توکن ها هم ورداست. اگر Π ماتریس جایگشت باشد:

$$\text{Attn}(\Pi X) = \Pi \text{Attn}(X) \quad (۲۶.۱۱)$$

در نتیجه مدل ترتیب شبکه تصویر را از محتوای توکن ها به تنهایی نمی داند. اطلاعات مکان باید به صورت مطلق، نسبی یا ضمنی افزوده شود.

۱.۳.۱۱ embedding مطلق آموختنی

در ViT اولیه، ماتریس $P \in \mathbb{R}^{(N+1) \times D}$ به توکن ها افزوده می شود:

$$Z_{\cdot} = [z_{\text{cls}}; Z_{\text{patch}}] + P. \quad (۲۷.۱۱)$$

مزیت آن سادگی است. ضعف آن وابستگی به grid آموزش است. برای وضوح جدید، بخش دوبعدی `em-bedding` به شبکه تبدیل و با interpolation تغییر اندازه می یابد. interpolation یک فرض `smoothness` تحمیل می کند و برای تغییر شدید نسبت تصویر یا `size patch` ممکن است artefact بسازد.

۲.۳.۱۱ کد سینوسی دوبعدی

می توان موقعیت سطر و ستون را جداگانه با سینوس/کسینوس کد کرد. برای بعد زوج:

$$PE_{(p, 2i)} = \sin(p / 10000^{2i/D}), \quad (۲۸.۱۱)$$

$$PE_{(p, 2i+1)} = \cos(p / 10000^{2i/D}). \quad (۲۹.۱۱)$$

در دوبعد، بخشی از کانال‌ها به x و بخشی به y اختصاص می‌یابد. این کد پارامتر ندارد و extrapolation طول را ممکن می‌کند، اما هندسه فرکانسی آن لزوماً برای همه داده‌ها بهینه نیست.

۳.۳.۱۱ bias مکانی نسبی

به جای افزودن بردار به توکن، bias وابسته به جابه‌جایی به attention logit افزوده می‌شود:

$$s_{ij} = \frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d_k}} + b_{\Delta x_{ij}, \Delta y_{ij}}. \quad (3.0.11)$$

در پنجره ثابت، جدول bias کوچک است. برای اندازه‌های متغیر می‌توان تابع پیوسته یا تجزیه محوری ساخت. bias نسبی برای انتقال مکانی طبیعی‌تر است، اما مرزها و clipping جابه‌جایی باید دقیق تعریف شوند.

۴.۳.۱۱ RoPE دوبعدی

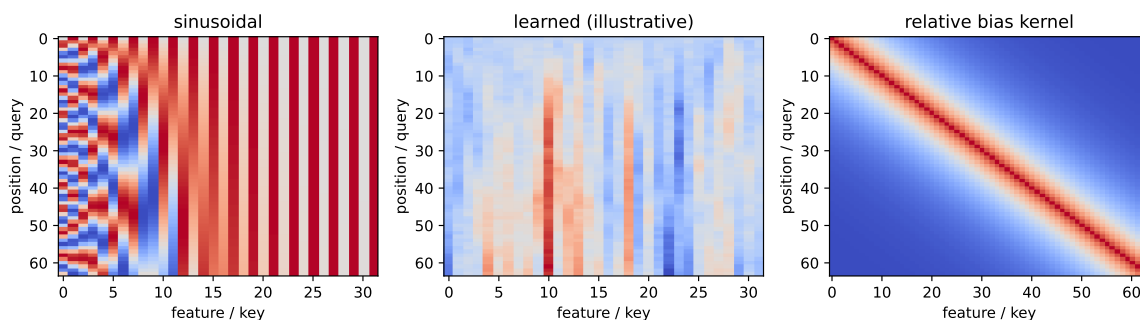
RoPE هر جفت مؤلفه query/key را با زاویه وابسته به موقعیت می‌چرخاند. در یک بعد:

$$\mathbf{q}'_p = \mathbf{R}(p)\mathbf{q}_p, \quad \mathbf{k}'_q = \mathbf{R}(q)\mathbf{k}_q, \quad (3.1.11)$$

و ضرب داخلی به اختلاف $q - p$ وابسته می‌شود:

$$(\mathbf{R}(p)\mathbf{q})^\top (\mathbf{R}(q)\mathbf{k}) = \mathbf{q}^\top \mathbf{R}(q - p)\mathbf{k}. \quad (3.2.11)$$

در تصویر، ابعاد به چرخش‌های مربوط به محور افقی و عمودی تقسیم می‌شوند. انتخاب فرکانس و scaling برای تغییر وضوح مهم است.



شکل ۹.۱۱: سه نمایش آموزشی از اطلاعات مکان: کد سینوسی، embedding آموختنی و kernel نسبی.

خطای رایج

کدگذاری مکان می‌تواند «سوگیری مکان» بسازد: مدل ممکن است اشیاء را بر اساس جای معمول در داده تشخیص دهد. آزمون انتقال، crop غیرمرکزی، shuffle tile و داده‌های بدون جهت ترجیحی برای ممیزی این مسئله ضروری‌اند.

۴.۱۱ Transformer Vision از ابتدا تا یک backbone عمومی

۱.۴.۱۱ معماری پایه ViT

یک ViT طبقه‌بندی معمولاً شامل مراحل زیر است:

۱. تبدیل تصویر به N توکن و وصله و projection به بعد D ؛
 ۲. افزودن توکن CLS و embedding مکان؛
 ۳. عبور از L بلوک transformer؛
 ۴. استفاده از توکن CLS یا میانگین توکن‌ها؛
 ۵. یک سر خطی برای وظیفه پایین‌دست.
- اگر نسبت MLP برابر r و تعداد سرها h باشد، تعداد پارامتر تقریبی هر بلوک، با صرف نظر از bias و normalization،

$$4D^2 + 2rD^2 = (4 + 2r)D^2 \quad (33.11)$$

است؛ بخش $4D^2$ از QKV و projection خروجی و بخش $2rD^2$ از دو لایه MLP می‌آید. بنابراین برخلاف تصور رایج، در بسیاری از ViTها MLP سهمی هم‌اندازه یا بزرگ‌تر از attention در پارامتر و محاسبه دارد. هزینه تقریبی یک بلوک:

$$C \approx 4ND^2 + 2N^2D + 2rND^2. \quad (34.11)$$

برای N کوچک، ها projection و MLP غالب‌اند؛ برای وضوح بالا، جمله N^2D غالب می‌شود. این تفکیک توضیح می‌دهد چرا کاهش توکن در مدل بزرگ همیشه همان سودی را ندارد که محاسبه فقط از روی N^2 وعده می‌دهد.

۲.۴.۱۱ توکن CLS یا pooling؟

توکن CLS یک query آموختنی است که باید اطلاعات لازم برای تصمیم را جمع‌آوری کند. میانگین‌گیری توکن‌ها، فرض می‌کند همه مکان‌ها سهمی متناسب دارند. انتخاب میان این دو به recipe و پیش‌آموزش وابسته است. در مدل‌های خودنظارتی، گاهی توکن CLS برای ویژگی جهانی و patch tokenها برای انتقال متراکم به کار می‌روند. گزارش فقط «خروجی backbone» کافی نیست؛ باید مشخص شود کدام لایه و کدام pooling استفاده شده است.

۳.۴.۱۱ ViT و نیاز به داده

ViT اولیه با پیش‌آموزش بزرگ مقیاس درخشید، زیرا locality سخت CNN را ندارد و باید بخشی از ساختار را از داده بیاموزد. اما نتیجه «ترنسفورم همیشه به صدها میلیون تصویر نیاز دارد» نادرست است. های‌ریس recipe قوی، augmentation، distillation regularization و pretraining self-supervised می‌توانند مدل را روی داده متوسط آموزش دهند. تفاوت اصلی، مقدار پیشین معماری و محل ورود آن است.

۴.۴.۱۱ DeiT: داده‌کارآمدی و توکن تقطیر

DeiT با recipe آموزشی دقیق و ViT distillation را روی ImageNet بدون پیش‌آموزش خارجی رقابتی کرد. توکن تقطیر z_{dist} در کنار CLS وارد دنباله می‌شود و یک سر جدا خروجی معلم را تقلید می‌کند. تابع هدف نمونه:

$$\mathcal{L} = \lambda \text{CE}(y, p_{\text{cls}}) + (1 - \lambda) \text{CE}(\hat{y}_T, p_{\text{dist}}). \quad (35.11)$$

این طراحی نشان می‌دهد token می‌تواند نقش یک رابط وظیفه‌ای داشته باشد، نه فقط نماینده یک وصله.

۵.۴.۱۱ هیبرید CNN-Transformer

CNN stem یا backbone اولیه، map feature محلی می‌سازد و transformer روابط دوربرد را مدل می‌کند. این طراحی در داده کم یا وظایف با بافت محلی قوی مفید است، اما تفسیر هزینه و resolution پیچیده‌تر می‌شود. مقایسه منصفانه باید، recipe تعداد پارامتر، FLOPs و latency یکسان را در نظر گیرد؛ نام «هیبرید» به تنهایی مزیت نیست.

۵.۱۱ معماری‌های سلسله‌مراتبی و attention محلی

۱.۵.۱۱ Transformer Vision Pyramid

PVT نمایش چندمقیاسی می‌سازد و تعداد توکن را در هر stage کاهش می‌دهد. atten- spatial-reduction tion کلید و value را روی شبکه کم‌وضوح‌تر محاسبه می‌کند؛ اگر نسبت کاهش R باشد، تعداد key/value تقریباً N/R^2 است و هزینه query-key به

$$\mathcal{O}\left(\frac{N^2 D}{R^2}\right) \quad (36.11)$$

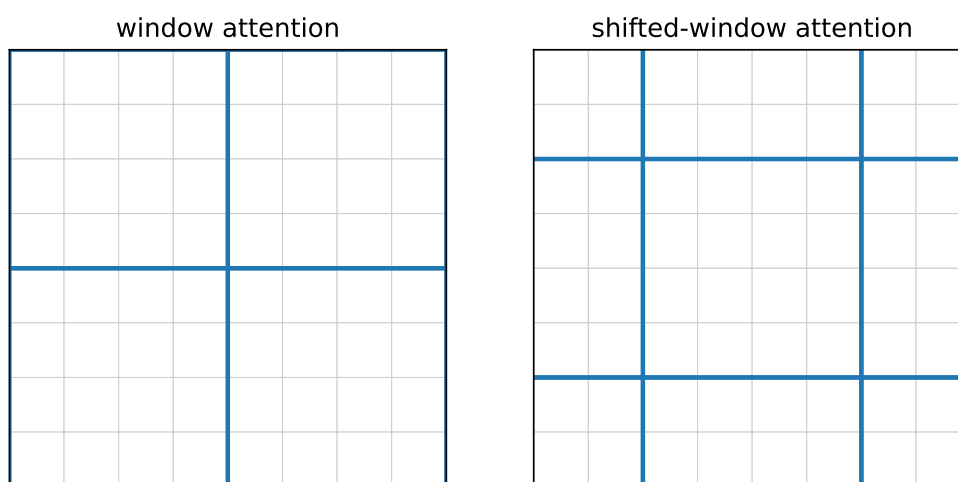
کاهش می‌یابد. کاهش شدید ممکن است جزئیات کوچک را در key/value حذف کند.

۲.۵.۱۱ Swin و پنجره جابه‌جاشونده

در Swin، attention داخل پنجره $M \times M$ انجام می‌شود. اگر تعداد کل توکن N باشد، هزینه تقریبی

$$\mathcal{O}(NM^2D) \quad (۳۷.۱۱)$$

است؛ نسبت به N خطی است و وقتی M ثابت باشد. اگر پنجره‌ها در همه لایه‌ها ثابت بمانند، تبادل اطلاعات میان پنجره‌ها محدود می‌شود. جابه‌جایی پنجره در بلوک بعدی، توکن‌هایی را که قبلاً در پنجره‌های جدا بودند نگار هم می‌آورد.



شکل ۱۰.۱۱: attention پنجره‌ای و جابه‌جایی پنجره برای ارتباط بین نواحی. در پیاده‌سازی واقعی، cyclic shift و mask مرزی استفاده می‌شود.

۳.۵.۱۱ Hierarchical attention pooling MViT و

MViT اندازه دنباله و بعد کانال را در طول شبکه تغییر می‌دهد و pooling را داخل سازوکار attention وارد می‌کند. Hierarchical نشان می‌دهد با پیش‌آموزش ماسک شده قوی، می‌توان بخش‌هایی از پیچیدگی معماری سلسله‌مراتبی را حذف و backbone ساده‌تر و سریع‌تری ساخت. پیام آموزشی مهم این است:

قدرت نهایی حاصل برهم‌کنش معماری و objective پیش‌آموزش است؛ نمی‌توان backbone را مستقل از روش آموزش رتبه‌بندی کرد.

۴.۵.۱۱ MaxViT و attention چندمحوری

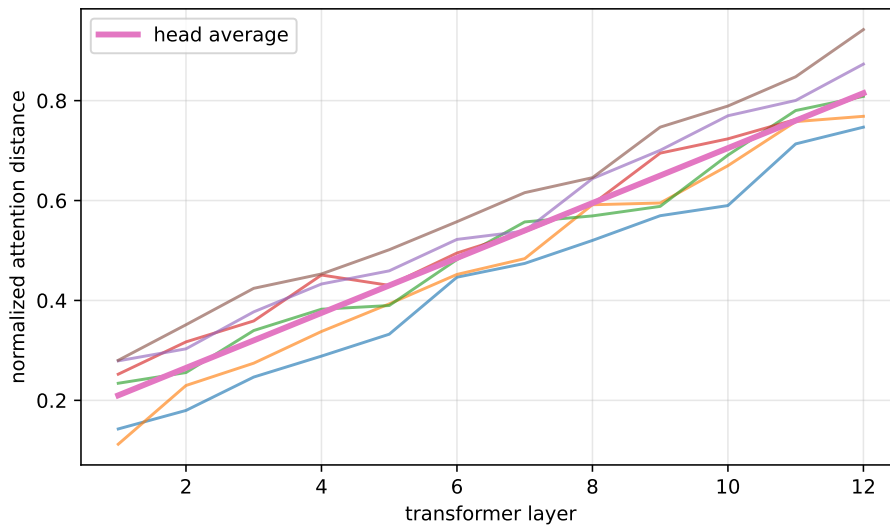
MaxViT attention محلی بلوکی را با attention شبکه‌ای یا dilated ترکیب می‌کند تا در هر بلوک هم تعامل محلی و هم مسیرهای دوربرد داشته باشد. اگر تنها پنجره محلی باشد، قطر گراف ارتباطی با عمق کاهش می‌یابد؛ محور دوم ارتباط سراسری را با هزینه خطی فراهم می‌کند.

۵.۵.۱۱ میدان تعامل و فاصله attention

در CNN میدان پذیرش با kernel و stride تعیین می‌شود. در attention سراسری، از اولین لایه همه توکن‌ها بالقوه مرتبطاند، اما وزن واقعی می‌تواند محلی باشد. فاصله میانگین attention برای سر h و لایه l را می‌توان نوشت:

$$D_{l,h} = \frac{1}{N} \sum_i \sum_j a_{ij}^{(l,h)} \|p_i - p_j\|_2. \quad (38.11)$$

این معیار ظرفیت نظری را با استفاده واقعی مقایسه می‌کند، ولی بدون تحلیل value و gradient کامل نیست.



شکل ۱۱.۱۱: نمونه آموزشی رشد فاصله attention با عمق؛ سرها می‌توانند رفتارهای متفاوتی داشته باشند.

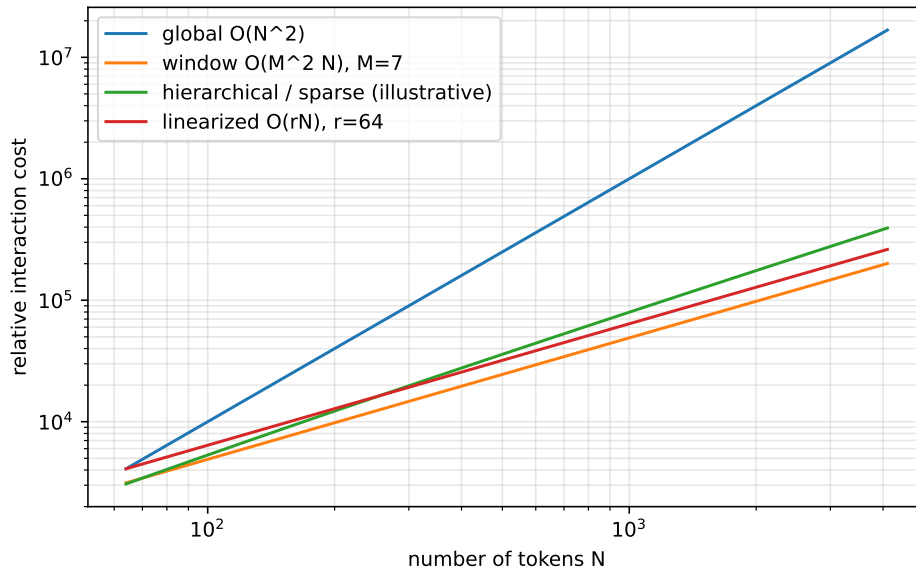
۶.۱۱ کارایی: attention، FLOPs، حافظه و حرکت داده

۱۰.۶.۱۱ مسئله درجه‌دو

ماتریس logits برای B batch، h سر و N توکن دارای BhN^2 عنصر است. در precision شانزده بیتی، فقط نگه‌داری یک tensor با $B = 32$ ، $h = 12$ و $N = 1024$ حدود

$$32 \times 12 \times 1024^2 \times 2 \text{ bytes} \approx 768 \text{ MiB} \quad (39.11)$$

نیاز دارد؛ بدون gradient و های tensor میانی. بنابراین مسئله attention فقط تعداد ضرب نیست، بلکه جابه‌جایی داده و materialization ماتریس است.



شکل ۱۲.۱۱: رشد هزینه تعامل برای چند خانواده attention. منحنی‌ها مقیاس مفهومی‌اند و ثابت‌های kernel و سخت‌افزار را نشان نمی‌دهند.

۲.۶.۱۱ خانواده‌های کاهش هزینه

۱. محلی/پنجره‌ای: هر query فقط زیرمجموعه‌ای از همسایه‌ها را می‌بیند؛
 ۲. پراکنده ساختاری: الگوی اتصال، block محوری، dilated یا global-token است؛
 ۳. کاهش key/value: query key/value کامل با key/value کم‌وضوح تعامل می‌کند؛
 ۴. کم‌رتبه یا kernelized: attention softmax با فاکتوربندی یا map feature تقریب زده می‌شود؛
 ۵. کاهش توکن: merging pruning یا TokenLearner طول دنباله را کم می‌کند؛
 ۶. الگوریتم دقیق حافظه‌کارآمد: خروجی exact با tiling و recomputation محاسبه می‌شود.
- هیچ روش واحدی برای همه اندازه‌ها و سخت‌افزارها برتر نیست. attention خطی ممکن است FLOPs کمتری داشته باشد، اما kernel نامناسب یا ثابت بزرگ، latency را بدتر کند.

۳.۶.۱۱ FlashAttention: تغییر الگوریتم، نه تقریب تابع

FlashAttention attention دقیق را با tiling محاسبه می‌کند تا خواندن/نوشتن حافظه پرهزینه کاهش یابد و ماتریس کامل $N \times N$ در حافظه اصلی materialize نشود. softmax به صورت بلوکی و پایدار با نگهداری ماکزیمم و مجموع نرمال‌ساز به روزرسانی می‌شود. برای دو بلوک، logits، اگر m_1, m_2 ماکزیمم‌های محلی باشند، ماکزیمم ترکیبی

$$m = \max(m_1, m_2) \quad (۴.۰.۱۱)$$

و مجموع نمایی ترکیبی با rescale:

$$\ell = e^{m_1 - m} \ell_1 + e^{m_2 - m} \ell_2 \quad (۴۱.۱۱)$$

محاسبه می‌شود. این هویت اجازه می‌دهد streaming softmax باشد.

کتابخانه و ابزار: SDPA PyTorch

تابع scaled_dot_product_attention یک رابط سطح بالا برای attention مقیاس شده است و بسته به dtype، device، شکل و mask می‌تواند backend متفاوتی انتخاب کند. برای benchmark باید warm-up، synchronization، forward/backward و dropout و طول دنباله واقعی ثبت شود. nn.MultiheadAttention در شرایط مناسب از همین های kernel بهره استفاده می‌کند.

کارگاه کد و آزمایش: benchmark کوچک attention

```

1 from __future__ import annotations
2 import argparse, time, json
3 import torch
4 from torch.nn import functional as F
5
6
7 def bench(n=256, d=64, heads=4, iters=20):
8     q=torch.randn(1,heads,n,d)
9     k=torch.randn_like(q); v=torch.randn_like(q)
10    for _ in range(3): F.scaled_dot_product_attention(q,k,v)
11    times=[]
12    for _ in range(iters):
13        t=time.perf_counter(); F.scaled_dot_product_attention(q,k,v);
14        times.append(time.perf_counter()-t)
15    return {'tokens':n, 'head_dim':d, 'heads':heads, 'median_ms':1000*
16           float(torch.tensor(times).median()),
17           'backend':'cpu' if not torch.cuda.is_available() else
18           torch.cuda.get_device_name(0)}
19
20 if __name__=='__main__':
21     p=argparse.ArgumentParser(); p.add_argument('--tokens',type=int,
22     default=128); args=p.parse_args()
23     result=bench(n=args.tokens,iters=5); print(json.dumps(result,
24     indent=2))

```

خروجی CPU این بسته فقط test smoke است. هیچ عدد آن نباید به عنوان benchmark عمومی گزارش شود. روی GPU باید torch.cuda.synchronize() و اندازه‌گیری memory peak افزوده شود.

۴.۶.۰۱۱ کاهش توکن و خطر سوگیری

اگر score توکن g_i باشد، pruning سخت توکن‌های $g_i < \tau$ را حذف می‌کند. در آموزش، تصمیم گسسته با mask نرم، straight-through Gumbel، یا loss بودجه تقریب زده می‌شود. یک تابع نمونه:

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda \left(\frac{1}{N} \sum_i m_i - \rho \right)^2. \quad (۴۲.۱۱)$$

توکن‌های مربوط به شیء کوچک ممکن است score پایین اما اهمیت بالا داشته باشند. برای کاربرد پزشکی یا صنعتی، ارزیابی باید بر حسب اندازه ضایعه/شیء و ناحیه‌های کم‌نمونه تفکیک شود.

۷.۰۱۱ یادگیری خودنظارتی: مسئله دقیق چیست؟

داده بدون برچسب $\{x_i\}_{i=1}^n$ داریم و می‌خواهیم encoder

$$h = f_{\theta}(x) \quad (۴۳.۱۱)$$

نمایشی بسازد که با هزینه برچسب کم به وظایف گوناگون منتقل شود. خودنظارتی از تبدیل‌ها، ماسک، زمان، چندوجهی بودن یا ساختار نمونه، هدف مصنوعی می‌سازد. کیفیت هدف با سه پرسش سنجیده می‌شود:

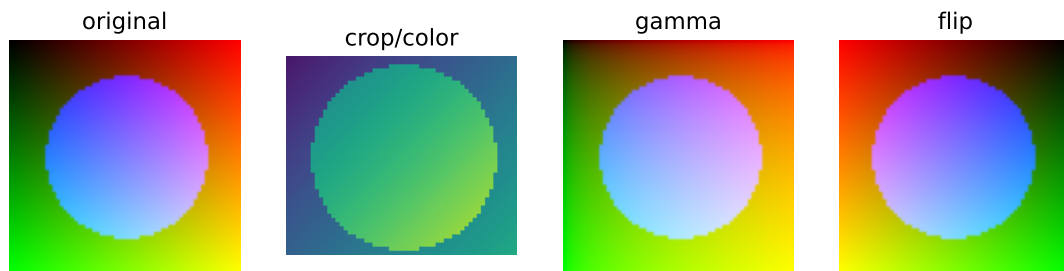
۱. کدام اطلاعات باید ناوردا شوند؟
۲. کدام اطلاعات باید حفظ شوند؟
۳. چه چیزی مانع پاسخ ثابت یا نمایش کم‌رتبه می‌شود؟

۱۰.۷.۰۱۱ دو نما و گروه تبدیل

اگر $t_1, t_2 \sim \mathcal{T}$ دو augmentation باشند، نماها

$$\tilde{x}^{(1)} = t_1(x), \quad \tilde{x}^{(2)} = t_2(x) \quad (۴۴.۱۱)$$

باید هویت معنایی مشترک داشته باشند. انتخاب $\mathcal{T}$ در واقع تعریف عملی «چه چیزی یکسان است» است. crop شدید برای طبقه‌بندی شیء مفید است، اما در پزشکی ممکن است هاه crop دو ناحیه با تشخیص متفاوت بسازند. تغییر رنگ برای اشیاء طبیعی می‌تواند مطلوب باشد، اما در پاتولوژی یا سنجش ازدور اطلاعات تشخیصی را حذف کند.



شکل ۱۳.۱۱: augmentation سیگنال نظارت را تعریف می‌کند. ترکیب نامناسب می‌تواند pair positive غلط بسازد.

اصل راهنما

در یادگیری خودنظارتی، augmentation یک مرحله تزئینی preprocessing نیست؛ بخشی از مدل آماری و تابع هدف است. هر مقاله باید rationale دامن‌های برای هر تبدیل ارائه کند.

۲۰۷.۱۱ سه فضای متفاوت

معمولاً encoder نمایش h ، head projection بردار $z = g(h)$ و سرپایین دست خروجی y را می‌سازند. loss خودنظارتی اغلب روی z اعمال می‌شود، اما انتقال از h انجام می‌شود. head projection می‌تواند اطلاعات لازم برای ناوردایی pretext را از فضای عمومی تر h جدا کند. حذف این تفکیک گاهی performance انتقال را کاهش می‌دهد.

۸.۱۱ یادگیری کنتراستی و InfoNCE

۱۰۸.۱۱ تابع هدف NT-Xent

برای batch شامل B تصویر و دو نمای هر تصویر، $2B$ embedding نرمال شده داریم. اگر i و j یک زوج مثبت باشند، loss جهت‌دار

$$\ell_{i \rightarrow j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k \neq i} \exp(\text{sim}(z_i, z_k)/\tau)} \quad (۴۵.۱۱)$$

است، که معمولاً

$$\text{sim}(a, b) = \frac{a^T b}{\|a\|_2 \|b\|_2} \quad (۴۶.۱۱)$$

و $\tau > 0$ دماست. loss نهایی میانگین دو جهت و همه هاست positive

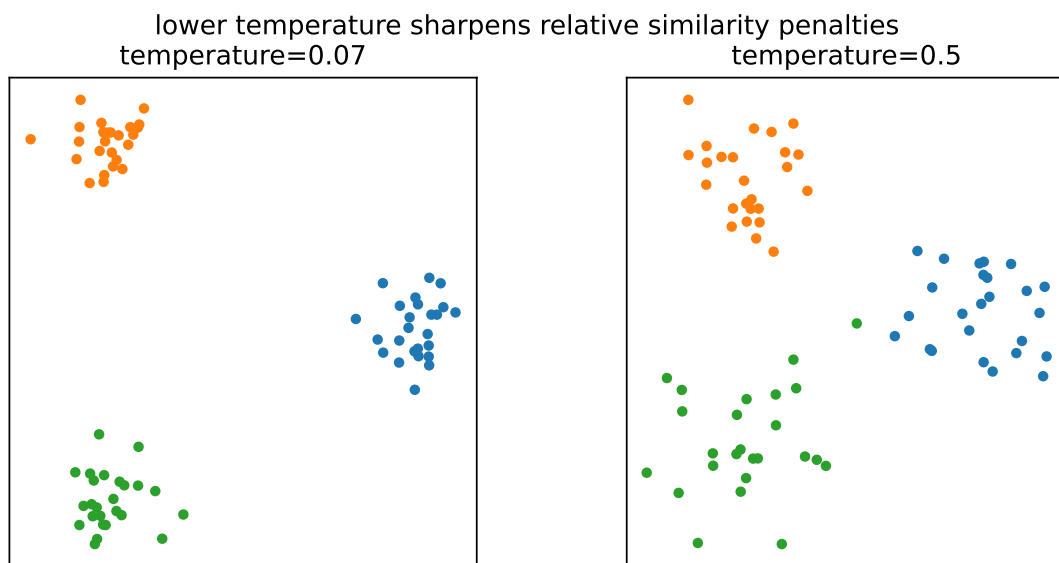
اگر هاتل را $u_{ik} = \text{sim}(z_i, z_k)/\tau$ و احتمال softmax را p_{ik} بنامیم:

$$\frac{\partial \ell_i}{\partial u_{ik}} = p_{ik} - [k = j]. \quad (۴۷.۱۱)$$

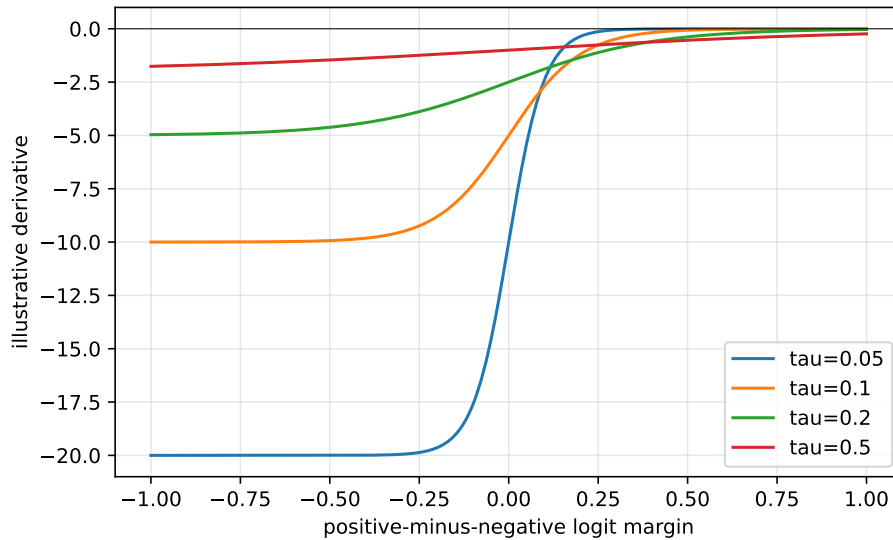
از آنجا که $u = s/\tau$:

$$\frac{\partial \ell_i}{\partial s_{ik}} = \frac{1}{\tau} (p_{ik} - [k = j]). \quad (۴۸.۱۱)$$

پس دمای کوچک هم توزیع را تیز و هم اندازه گرادیان را بزرگ می‌کند. دمای بسیار کم می‌تواند روی های negative سخت یا حتی false ها negative تمرکز افراطی ایجاد کند.



شکل ۱۴.۱۱: نمای مفهومی هندسه کنتراستی و اثر دما بر فشردگی نسبی خوشه‌ها.



شکل ۱۵.۱۱: نمونه آموزشی اثر دما بر مشتق margin. دمای پایین حساسیت را زیاد می‌کند، اما پایداری و negative false را دشوارتر می‌سازد.

۲۰.۸.۱۱ unfirmity و alignment

دو فشار هندسی در روش کنتراستی وجود دارد:

۱. هم‌ترازی: نمایش دو نمای یک نمونه نزدیک شود؛

۲. پراکندگی: نمایش نمونه‌های متفاوت روی کره collapse نکند.

یک معیار هم‌ترازی:

$$\mathcal{L}_{\text{align}} = \mathbb{E} \|z - z^+\|_p^2 \quad (۴۹.۱۱)$$

و معیار یکنواختی نمونه:

$$\mathcal{L}_{\text{unif}} = \log \mathbb{E}_{i \neq j} \exp(-t \|z_i - z_j\|_p^2) \quad (۵۰.۱۱)$$

است. loss پایین به تنهایی کافی نیست؛ ممکن است augmentation یا مجموعه هانگاتیو negative وظیفه‌ای بسازد که با انتقال پایین دست ناسازگار باشد.

۳۰.۸.۱۱ ارتباط با کران اطلاعات متقابل و احتیاط لازم

InfoNCE در شرایط خاص کران پایینی برای اطلاعات متقابل میان نماها فراهم می‌کند. برای K نمونه در dictionary، شکلی از رابطه

$$I(X; Y) \geq \log K - \mathcal{L}_{\text{NCE}} \quad (۵۱.۱۱)$$

به دست می‌آید. اما این رابطه نباید به ادعای ساده «کمینه کردن loss یعنی پیشینه کردن تمام اطلاعات مفید» تبدیل شود. نوع score توزیع، negative ظرفیت critic و augmentation تعیین می‌کنند چه اطلاعاتی باقی بماند. در عمل هدف، اطلاعات متقابل خام پیکسل نیست؛ ناوردایی کنترل شده و انتقال پذیری است.

۴.۸.۱۱ SimCLR

SimCLR نشان داد ترکیب augmentation قوی، head projection غیرخطی، batch بزرگ و آموزش طولانی می‌تواند بدون bank memory نمایش قدرتمند بسازد. خط لوله:

$$x \xrightarrow{t_1, t_2} \tilde{x}^{(1)}, \tilde{x}^{(2)} \xrightarrow{f_\theta} h^{(1)}, h^{(2)} \xrightarrow{g_\phi} z^{(1)}, z^{(2)}. \quad (52.11)$$

Batch بزرگ تعداد negative را زیاد می‌کند، اما نیاز حافظه و حساسیت به all-gather distributed را افزایش می‌دهد. اگر embedding از همه GPUها جمع می‌شود، gradient و ordering ها positive باید درست مدیریت شود.

کارگاه کد و آزمایش: پیاده سازی InfoNCE و یک گام SimCLR

```

1 from __future__ import annotations
2 import torch
3 from torch.nn import functional as F
4
5
6 def nt_xent(z1: torch.Tensor, z2: torch.Tensor, temperature: float =
7     0.2) -> torch.Tensor:
8     """Symmetric InfoNCE / NT-Xent for two aligned batches."""
9     if z1.shape != z2.shape or z1.ndim != 2:
10         raise ValueError('z1 and z2 must have equal shape [B,D]')
11     if temperature <= 0:
12         raise ValueError('temperature must be positive')
13     z1, z2 = F.normalize(z1, dim=1), F.normalize(z2, dim=1)
14     logits12 = z1 @ z2.T / temperature
15     labels = torch.arange(z1.shape[0], device=z1.device)
16     return 0.5 * (F.cross_entropy(logits12, labels) + F.cross_entropy(
17         logits12.T, labels))
18
19 if __name__ == '__main__':
20     torch.manual_seed(2)
21     base = torch.randn(16, 64)
22     good = nt_xent(base + .02*torch.randn_like(base), base + .02*torch.
23         randn_like(base))
24     bad = nt_xent(torch.randn_like(base), torch.randn_like(base))
25     print({'aligned_loss': float(good), 'random_loss': float(bad)})
26     assert good < bad

```

```

1 from __future__ import annotations
2 import torch
3 from torch import nn
4 from torch.nn import functional as F
5 from info_nce import nt_xent
6
7
8 class SmallEncoder(nn.Module):
9     def __init__(self, out_dim=128):
10         super().__init__()
11         self.net = nn.Sequential(
12             nn.Conv2d(3, 32, 3, padding=1), nn.BatchNorm2d(32), nn.
ReLU(),
13             nn.Conv2d(32, 64, 3, stride=2, padding=1), nn.BatchNorm2d
(64), nn.ReLU(),
14             nn.Conv2d(64, 128, 3, stride=2, padding=1), nn.BatchNorm2d
(128), nn.ReLU(),
15             nn.AdaptiveAvgPool2d(1), nn.Flatten())
16         self.proj = nn.Sequential(nn.Linear(128, 256), nn.ReLU(), nn.
Linear(256, out_dim))
17
18     def forward(self, x):
19         h = self.net(x)
20         return h, self.proj(h)
21
22
23 def augment(x):
24     # Educational tensor-only augmentation; use torchvision v2 for
real training.
25     if torch.rand(()) < .5:
26         x = x.flip(-1)
27     gain = torch.empty(x.shape[0], 1, 1, 1, device=x.device).uniform_
(.7, 1.3)
28     noise = .04 * torch.randn_like(x)
29     return (x * gain + noise).clamp(-3, 3)
30
31
32 if __name__ == '__main__':
33     torch.manual_seed(3)
34     model = SmallEncoder()
35     opt = torch.optim.AdamW(model.parameters(), lr=1e-3)
36     x = torch.randn(32, 3, 32, 32)
37     _, z1 = model(augment(x)); _, z2 = model(augment(x))
38     loss = nt_xent(z1, z2, .2)
39     opt.zero_grad(); loss.backward(); opt.step()
40     print({'one_step_loss': float(loss.detach()), 'embedding_shape':

```

41

```
tuple(z1.shape)})
assert torch.isfinite(loss)
```

این کد آموزشی augmentation محدودی دارد. برای پژوهش واقعی، torchvision.transforms.v2 یا Kornia، ثبت پارامتر تبدیل و آزمون دامنه‌ای لازم است.

۵.۸.۱۱ dictionary MoCo: پویا و encoder تکانه‌ای

MoCo صافی از key های batch گذشته می‌سازد تا dictionary بزرگ بدون batch عظیم به دست آید. encoder کلید با EMA به‌روزرسانی می‌شود:

$$\theta_k \leftarrow m\theta_k + (1 - m)\theta_q. \quad (۵۳.۱۱)$$

اگر m کوچک باشد، key صف سریع ناسازگار می‌شوند؛ اگر بسیار نزدیک یک باشد، معلم دیر سازگار می‌شود. صف باید first-in-first-out باشد و key جاری پس از محاسبه loss وارد شوند. در نسخه‌های بعد، recipe و ViT backbone نقش بیشتری پیدا کردند.

۶.۸.۱۱ negative false و داده با ساختار گروهی

در batch، دو تصویر متفاوت ممکن است یک کلاس یا هویت معنایی داشته باشند و به‌اشتباه negative شوند. در داده پزشکی، دو crop از یک بیمار یا دو فریم نزدیک یک ویدئو نمونه آشکار است. راهکارها شامل metadata-aware، sampling حذفی negative مشکوک، contrastive supervised در بخش برجسب‌دار، multi-positive و روش‌های بدون negative است. اما حذف negative بدون سازوکار دیگر می‌تواند collapse ایجاد کند.

۹.۱۱ خوشه‌بندی آنلاین و SwAV

SwAV به‌جای مقایسه همه زوج‌ها، assignment دو نمای یک تصویر به هاپروتوتایپ را سازگار می‌کند. اگر هاپروتوتایپ ستون‌های $C \in \mathbb{R}^{D \times K}$ باشند:

$$p = \text{softmax}(C^T z / \tau). \quad (۵۴.۱۱)$$

Assignment هدف q با قید استفاده متوازن از هاپروتوتایپ و الگوریتم Sinkhorn-Knopp محاسبه می‌شود. cross-view: loss

$$\mathcal{L} = - \sum_k q_k^{(1)} \log p_k^{(2)} - \sum_k q_k^{(2)} \log p_k^{(1)}. \quad (۵۵.۱۱)$$

قید تعادل مانع می‌شود همه نمونه‌ها به یک prototype بروند، اما اگر داده واقعاً نامتوازن باشد، تعادل مصنوعی می‌تواند ساختار طبیعی را تحریف کند.

الگوریتم مفهومی Sinkhorn برای assignment متوازن

۱. های prototype score را نمایی کنید و ماتریس Q بسازید؛
۲. چند بار سطرها را برای مجموع برابر $1/K$ نرمال کنید؛
۳. ستون‌ها را برای مجموع برابر $1/B$ نرمال کنید؛
۴. assignment نهایی را به مقیاس batch برگردانید؛
۵. gradient از teacher assignment عبور ندهید.

۱۰.۱۱ روش‌های بدون negative صریح: DINO و SimSiam BYOL

۱۰.۱۰.۱۱ BYOL و نقش عدم تقارن

در BYOL، شبکه online شامل projector encoder و predictor است؛ شبکه encoder target و projector دارد و با EMA به‌روزرسانی می‌شود. loss نمونه پس از نرمال‌سازی:

$$\mathcal{L}_{\text{BYOL}} = 2 - 2 \frac{q_{\theta}(z_1)^T z_2^{\text{target}}}{\|q_{\theta}(z_1)\| \|z_2^{\text{target}}\|} + \text{sym.} \quad (۵۶.۱۱)$$

stop-gradient روی target و predictor تنها در شاخه online تقارن کامل را می‌شکند. توضیح دقیق ضد collapse هنوز به normalization optimization و معماری وابسته است؛ بیان اینکه EMA «به‌تنهایی مانع collapse است» بیش‌ازحد ساده است.

۲۰.۱۰.۱۱ SimSiam

SimSiam نشان داد حتی بدون encoder تکرانه‌ای نیز stop-gradient و predictor می‌تواند کافی باشند. این نتیجه اهمیت دینامیک بهینه‌سازی و عدم تقارن را برجسته کرد. در پیاده‌سازی، گذاشتن stop-gradient در محل اشتباه یا حذف predictor می‌تواند رفتار را تغییر دهد.

۳۰.۱۰.۱۱ DINO: خودتقطیر با centering و sharpening

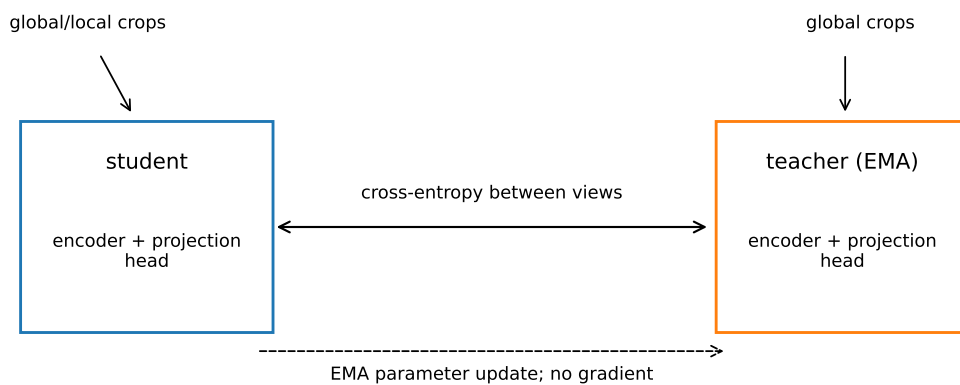
در DINO، دانش‌آموز چند crop و معلم معمولاً های crop سراسری را می‌بیند. توزیع دانش‌آموز:

$$p_s^{(k)}(x) = \frac{\exp(g_s(x)^{(k)}/\tau_s)}{\sum_{k'} \exp(g_s(x)^{(k')}/\tau_s)}, \quad (۵۷.۱۱)$$

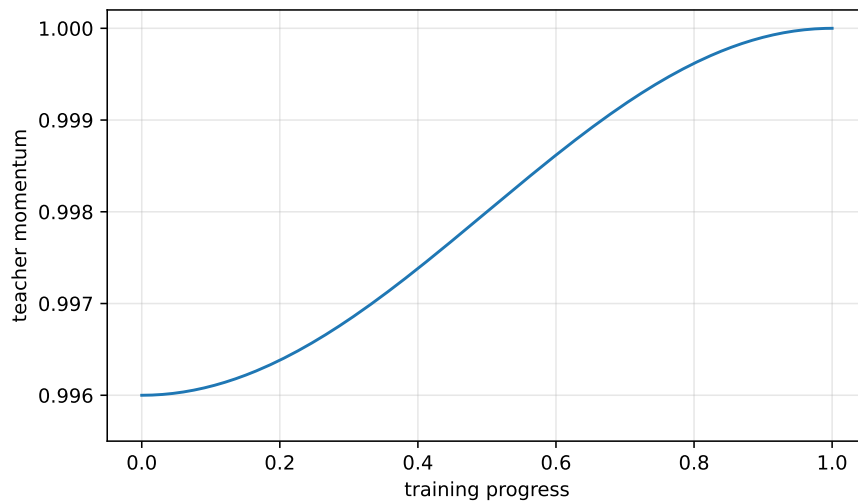
و معلم با center c :

$$p_t^{(k)}(x) = \frac{\exp((g_t(x)^{(k)} - c_k)/\tau_t)}{\sum_{k'} \exp((g_t(x)^{(k')} - c_{k'})/\tau_t)}. \quad (58.11)$$

، cross-entropy loss خروجی معلم و دانش آموز روی نماهای متفاوت است. τ_t کوچک sharpening ایجاد می کند؛ center با میانگین متحرک به روزرسانی می شود تا یک یا چند بعد غالب نشوند. معلم با EMA وزن ها ساخته می شود.



شکل ۱۶.۱۱: ساختار teacher-student در gradient DINO. تنها به دانش آموز می رسد و معلم با EMA تغییر می کند.



شکل ۱۷.۱۱: schedule نمونه برای افزایش momentum معلم به سمت یک. مقدار دقیق باید با batch، طول آموزش و مدل هماهنگ شود.

کارگاه کد و آزمایش: معلم EMA و loss تقطیر

```

1 from __future__ import annotations
2 import copy
3 import math
4 import torch
5 from torch import nn
6 from torch.nn import functional as F
7
8
9 @torch.no_grad()
10 def update_teacher(student: nn.Module, teacher: nn.Module, momentum:
    float):
11     for ps, pt in zip(student.parameters(), teacher.parameters()):
12         pt.data.mul_(momentum).add_(ps.data, alpha=1.0 - momentum)
13
14
15 def dino_loss(student_logits, teacher_logits, student_temp=.1,
    teacher_temp=.04, center=None):
16     if center is None:
17         center = torch.zeros(1, teacher_logits.shape[-1], device=
            teacher_logits.device)
18     p_t = F.softmax((teacher_logits - center) / teacher_temp, dim=-1).
        detach()
19     log_p_s = F.log_softmax(student_logits / student_temp, dim=-1)
20     return -(p_t * log_p_s).sum(dim=-1).mean()
21
22
23 class MLP(nn.Module):
24     def __init__(self, d=64, out=256):
25         super().__init__(); self.net=nn.Sequential(nn.Linear(d,128),nn.
            GELU(),nn.Linear(128,out))
26     def forward(self,x): return self.net(x)
27
28
29 if __name__ == '__main__':
30     torch.manual_seed(5)
31     student = MLP(); teacher = copy.deepcopy(student)
32     for p in teacher.parameters(): p.requires_grad_(False)
33     opt = torch.optim.AdamW(student.parameters(), 1e-3)
34     x1, x2 = torch.randn(32,64), torch.randn(32,64)
35     s = student(x1)
36     with torch.no_grad(): t = teacher(x2)
37     loss=dino_loss(s,t)
38     opt.zero_grad(); loss.backward(); opt.step(); update_teacher(
        student,teacher,.996)

```

```

39     delta=sum((a-b).abs().mean() for a,b in zip(student.parameters(),
teacher.parameters()))
40     print({'loss':float(loss.detach()), 'student_teacher_mean_delta':
float(delta.detach())})
41     assert torch.isfinite(loss)

```

نسخه کامل DINO به centering multi-crop توزیع شده، schedule دمای معلم، nor-weight malization سر و هماهنگی EMA میان پردازش‌ها نیاز دارد.

۴.۱۰.۱۱ چرا ویژگی‌های متراکم در DINO مهم شدند؟

در ViT خودنظارتی، patch‌ها token بدون برچسب پیکسلی می‌توانند مرز و جزء معنایی را آشکار کنند. این خاصیت سبب شد های DINO backbone برای قطعه‌بندی، تطبیق، عمق، بازیابی و correspondence به‌کار روند. با این حال، کیفیت feature dense باید با benchmark متراکم سنجیده شود؛ دقت kNN روی CLS تضمین‌کننده مرز دقیق نیست.

۱۱.۱۱ کاهش افزونگی: VICReg و Twins Barlow

Twins Barlow ۱.۱۱.۱۱

دو embedding batch استاندارد شده $Z^A, Z^B \in \mathbb{R}^{B \times D}$ داریم. ماتریس همبستگی متقاطع:

$$C_{ij} = \frac{\sum_b Z_{bi}^A Z_{bj}^B}{\sqrt{\sum_b (Z_{bi}^A)^2} \sqrt{\sum_b (Z_{bj}^B)^2}}. \quad (۵۹.۱۱)$$

تابع هدف:

$$\mathcal{L}_{BT} = \sum_i (1 - C_{ii})^2 + \lambda \sum_{i \neq j} C_{ij}^2. \quad (۶۰.۱۱)$$

قطر، ناوردایی دو نما را تحمیل می‌کند و خارج قطر، افزونگی ابعاد را کاهش می‌دهد. استانداردسازی batch بخش مهم objective است؛ batch کوچک تخمین همبستگی را پرنویز می‌کند.

VICReg ۲.۱۱.۱۱

VICReg سه جمله را صریح می‌کند. جمله ناوردایی:

$$s(Z^A, Z^B) = \frac{1}{B} \sum_b \|z_b^A - z_b^B\|_2^2. \quad (۶۱.۱۱)$$

جمله واریانس برای هر شاخه:

$$v(\mathbf{Z}) = \frac{1}{D} \sum_j \max \left(0, \gamma - \sqrt{\text{Var}(\mathbf{Z}_{:,j}) + \epsilon} \right). \quad (۶۲.۱۱)$$

جمله کوواریانس:

$$c(\mathbf{Z}) = \frac{1}{D} \sum_{i \neq j} \text{Cov}(\mathbf{Z})_{ij}^2. \quad (۶۳.۱۱)$$

و loss:

$$\mathcal{L}_{\text{VICReg}} = \lambda s + \mu [v(\mathbf{Z}^A) + v(\mathbf{Z}^B)] + \nu [c(\mathbf{Z}^A) + c(\mathbf{Z}^B)]. \quad (۶۴.۱۱)$$

جمله واریانس جلوی collapse هر بعد را می‌گیرد؛ جمله کوواریانس ابعاد تکراری را کاهش می‌دهد. hyper-parameterها مقیاس سه جمله را متعادل می‌کنند و باید جداگانه log شوند.

مقایسه ضد فروپاشی

- InfoNCE: رقابت positive با ها؛ negative
- SwAV: assignment متوازن به ها؛ prototype
- BYOL/SimSiam: عدم تقارن، predictor و stop-gradient
- DINO: EMA، teacher centering و sharpening
- Barlow Twins: همانی کردن همبستگی متقاطع؛
- VICReg: کران پایین واریانس و جریمه کوواریانس؛
- MAE: هدف بازسازی غیرثابت برای هر تصویر و mask.

۱۲.۱۱ فروپاشی نمایش و ابزارهای تشخیص

۱۰۱۲.۱۱ فروپاشی کامل و فروپاشی بعدی

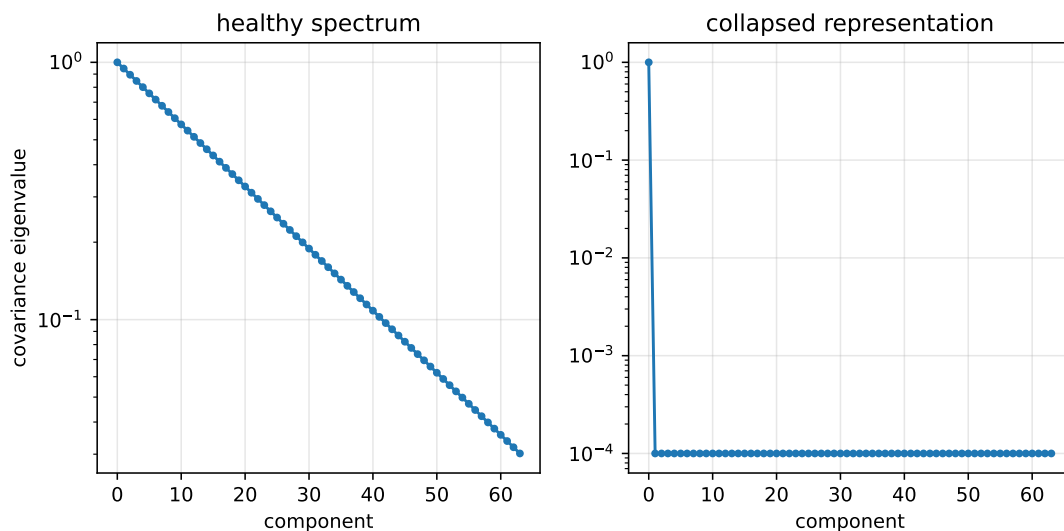
در collapse کامل:

$$f_{\theta}(x) = \mathbf{c} \quad \forall x. \quad (۶۵.۱۱)$$

اما collapse می‌تواند جزئی باشد: embedding تنها در زیرفضای کم‌بعد تغییر کنند. اگر ماتریس centered embedding برابر Z_c و مقادیر ویژه کوواریانس $\lambda_1 \geq \dots \geq \lambda_D$ باشند، rank مؤثر:

$$r_{\text{eff}} = \exp \left(- \sum_i p_i \log p_i \right), \quad p_i = \frac{\lambda_i}{\sum_j \lambda_j} \quad (۶۶.۱۱)$$

است. همچنین می‌توان تعداد ابعادی را که درصد مشخصی از انرژی را توضیح می‌دهند گزارش کرد.



شکل ۱۸.۱۱: طیف کوواریانس سالم و فروپاشیده. فقط norm متوسط embedding برای تشخیص collapse کافی نیست.

۲۰۱۲.۱۱ چک‌لیست پایش در آموزش

۱. انحراف معیار هر بعد embedding و کمینه آن؛
 ۲. spectrum کوواریانس و rank مؤثر؛
 ۳. entropy خروجی؛ teacher/student؛
 ۴. norm گرادیان projector و backbone؛
 ۵. میانگین شباهت positive و negative؛
 ۶. assignment histogram؛ prototype؛
 ۷. های‌loss جداگانه واریانس/کوواریانس/ناوردایی؛
 ۸. kNN یا probe linear سبک روی های‌snapshot دوره‌ای.
- Loss ثابت یا کاهش‌یابنده می‌تواند هم‌زمان با فروپاشی رخ دهد؛ diagnostic باید مستقیماً هندسه نمایش را بسنجد.

۱۳.۱۱ مدل سازی ماسک شده تصویر

۱۰۱۳.۱۱ صورت بندی عمومی

مجموعه شاخص توکن ها $\Omega = \{1, \dots, N\}$ را به visible و masked تقسیم می کنیم:

$$\Omega = \mathcal{V} \cup \mathcal{M}, \quad \mathcal{V} \cap \mathcal{M} = \emptyset. \quad (۶۷.۱۱)$$

مدل از $x_{\mathcal{V}}$ هدف $y_{\mathcal{M}}$ را پیش بینی می کند:

$$\hat{y}_{\mathcal{M}} = g_{\phi}(f_{\theta}(x_{\mathcal{V}}), \mathcal{M}). \quad (۶۸.۱۱)$$

تفاوت روش ها در تعریف y ، نسبت و شکل mask، معماری decoder و محل اعمال loss است.

۲۰۱۳.۱۱ BEiT و هدف گسسته

BEiT تصویر را دو بار می بیند: وصله های پیوسته برای encoder و توکن های گسسته حاصل از tokenizer خارجی برای هدف. اگر $q_i \in \{1, \dots, K\}$ که وصله i باشد:

$$\mathcal{L}_{\text{BEiT}} = - \sum_{i \in \mathcal{M}} \log p_{\theta}(q_i | x_{\mathcal{V}}). \quad (۶۹.۱۱)$$

مزیت، هدف طبقه بندی معنایی تر از regression پیکسل است؛ ضعف، وابستگی به tokenizer و pipeline چند مرحله ای. اگر tokenizer بافت یا رنگی را کد کند که برای وظیفه هدف نامطلوب است، همان bias را می آموزد.

۳۰۱۳.۱۱ MAE و بازسازی پیکسل

MAE فقط توکن های visible را وارد encoder سنگین می کند. سپس latent ها با mask ها token به ترتیب اصلی بازگردانده و decoder سبک پیکسل وصله های masked را بازسازی می کند. برای وصله i :

$$\mathcal{L}_{\text{MAE}} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \|\hat{x}_i - x_i\|_2^2. \quad (۷۰.۱۱)$$

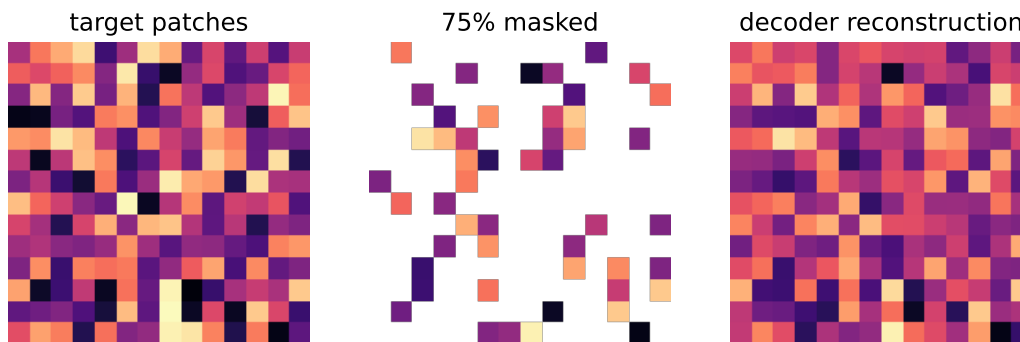
اغلب target هر وصله نرمال می شود:

$$\tilde{x}_i = \frac{x_i - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}}, \quad (۷۱.۱۱)$$

تا مدل کمتر روی روشنایی مطلق و بیشتر روی ساختار تمرکز کند.

نسبت mask بالا مانند ۷۵٪ دو اثر دارد: وظیفه را دشوار و redundancy محلی را کم می کند؛ و encoder را ارزان می سازد چون فقط ۲۵٪ توکن ها را می بیند. اگر نسبت خیلی پایین باشد، interpolation

محلی ساده کافی است؛ اگر بسیار بالا باشد، context برای ساختار ظریف ناکافی می‌شود.



شکل ۱۹.۱۱: ساختار مفهومی MAE با mask زیاد و decoder بازسازی. شکل داده مصنوعی است.

کارگاه کد و آزمایش: MAE آموزشی

```

1 from __future__ import annotations
2 import torch
3 from torch import nn
4 from patchify import patchify
5
6
7 def random_mask(tokens: torch.Tensor, ratio: float):
8     b, n, d = tokens.shape
9     keep = max(1, int(n * (1 - ratio)))
10    noise = torch.rand(b, n, device=tokens.device)
11    ids_shuffle = noise.argsort(dim=1)
12    ids_restore = ids_shuffle.argsort(dim=1)
13    ids_keep = ids_shuffle[:, :keep]
14    visible = torch.gather(tokens, 1, ids_keep[..., None].expand(-1,
15    -1, d))
16    mask = torch.ones(b, n, device=tokens.device)
17    mask[:, :keep] = 0
18    mask = torch.gather(mask, 1, ids_restore)
19    return visible, mask, ids_restore
20
21 class TinyMAE(nn.Module):
22     def __init__(self, patch=4, dim=96, decoder_dim=64):
23         super().__init__()
24         self.patch = patch
25         patch_dim = 3 * patch * patch
26         self.embed = nn.Linear(patch_dim, dim)
27         layer = nn.TransformerEncoderLayer(dim, 4, 4*dim, batch_first=
28         True, norm_first=True)
29         self.encoder = nn.TransformerEncoder(layer, 2)
30         self.to_dec = nn.Linear(dim, decoder_dim)

```

```

30     self.mask_token = nn.Parameter(torch.zeros(1, 1, decoder_dim))
31     dec = nn.TransformerEncoderLayer(decoder_dim, 4, 4*decoder_dim
, batch_first=True, norm_first=True)
32     self.decoder = nn.TransformerEncoder(dec, 1)
33     self.pred = nn.Linear(decoder_dim, patch_dim)
34
35     def forward(self, x, ratio=.75):
36         target = patchify(x, self.patch)
37         tokens = self.embed(target)
38         visible, mask, restore = random_mask(tokens, ratio)
39         encoded = self.encoder(visible)
40         dec_vis = self.to_dec(encoded)
41         b, n, _ = target.shape
42         missing = n - dec_vis.shape[1]
43         seq = torch.cat([dec_vis, self.mask_token.expand(b, missing,
-1)], dim=1)
44         seq = torch.gather(seq, 1, restore[... , None].expand(-1, -1,
seq.shape[-1]))
45         pred = self.pred(self.decoder(seq))
46         loss = ((pred - target) ** 2).mean(dim=-1)
47         loss = (loss * mask).sum() / mask.sum().clamp_min(1)
48         return loss, pred, mask
49
50
51 if __name__ == '__main__':
52     torch.manual_seed(4)
53     model = TinyMAE()
54     x = torch.randn(8, 3, 32, 32)
55     loss, pred, mask = model(x)
56     loss.backward()
57     print({'loss': float(loss.detach()), 'prediction': tuple(pred.
shape), 'masked_fraction': float(mask.mean().detach())})
58     assert torch.isfinite(loss)

```

این نسخه برای سادگی positional embedding ندارد. در مدل واقعی، ترتیب توکن visible، restoration، embedding index مکان encoder و decoder و schedule mask باید با دقت کنترل شوند.

۴.۱۳.۱۱ SimMIM

SimMIM نشان داد طراحی ساده mask و سر prediction سبک می‌تواند مؤثر باشد. پیام روش‌شناختی آن است که پیچیدگی tokenizer یا decoder همیشه ضروری نیست؛ اما نوع backbone، اندازه patch و target normalization نتیجه را تغییر می‌دهند.

۵.۱۳.۱۱ tokenizer iBOT: آنلاین

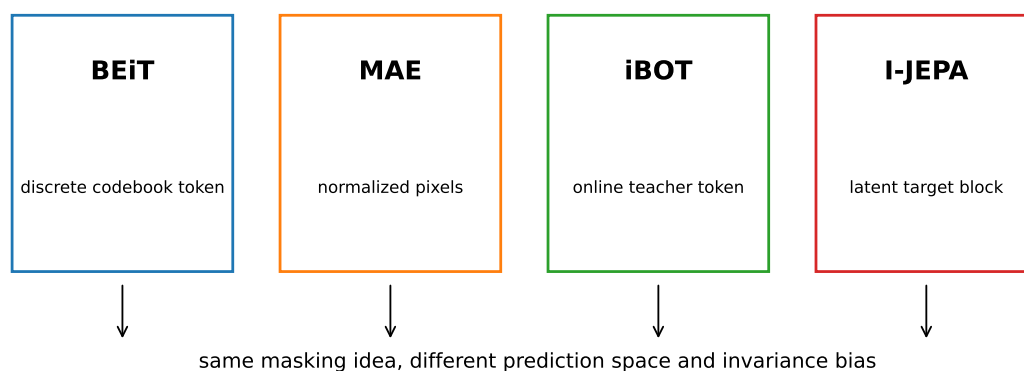
iBOT معلم را به tokenizer آنلاین تبدیل می‌کند. self-distillation هم روی token جهانی و هم masked token patch اعمال می‌شود. در نتیجه هدف محلی هم‌زمان با encoder تکامل می‌یابد و pipeline tokenizer جدا حذف می‌شود. اما ثبات معلم، centering و mask schedule پیچیده‌تر می‌گردد.

۶.۱۳.۱۱ latent contextualized هدف vec:2data

student data2vec ورودی masked را می‌بیند و نمایش contextualized معلم از ورودی کامل را پیش‌بینی می‌کند. اگر y_i میانگین یا ترکیب چند لایه معلم باشد:

$$\mathcal{L} = \sum_{i \in \mathcal{M}} \rho(\hat{y}_i - y_i), \quad (۷۲.۱۱)$$

که ρ می‌تواند smooth L_1 باشد. هدف نه‌کد گسسته و نه پیکسل است؛ context کامل معلم را فشرده می‌کند.



شکل ۲۰.۱۱: فضاهای هدف متفاوت در مدل‌سازی ماسک‌شده و پیش‌بینی نهفته. انتخاب target نوع اطلاعات آموخته‌شده را تعیین می‌کند.

۷.۱۳.۱۱ mask شکل

mask مستقل تصادفی، بلوکی، چندمقیاسی یا معنایی می‌تواند استفاده شود. mask تصادفی coverage یکنواخت دارد؛ mask بلوکی context دوربرد و ساختار کلان را می‌آزماید؛ mask معنایی خطر leakage از detector یا مدل قبلی دارد. برای داده پزشکی سه‌بعدی، mask مکعبی ممکن است ضایعه کوچک را کامل حذف کند و هدف بسیار مبهم شود.

چه چیزی را بازسازی کنیم؟

- پیکسل: دقیق و ساده، اما حساس به جزئیات کم‌اهمیت؛
- پیکسل نرمال‌شده: بافت و ساختار نسبی؛
- توکن گسسته: هدف طبقه‌بندی، وابسته به codebook؛
- ویژگی teacher: هدف معنایی‌تر، وابسته به کیفیت و collapse معلم؛
- feature چندمقیاسی: انتقال dense بهتر، هزینه حافظه بیشتر.

۱۴.۱۱ I-JEPA و پیش‌بینی در فضای نهفته

I-JEPA از یک context encoder، block نمایش چند block target را پیش‌بینی می‌کند. context encoder تنها پیکسل‌های context را می‌بیند؛ target encoder از تصویر کامل یا target نمایش می‌سازد و با EMA به‌روزرسانی می‌شود. predictor موقعیت target را نیز دریافت می‌کند:

$$\hat{y}_T = p_\phi(f_\theta(x_c), \text{pos}(\mathcal{T})). \quad (۷۳.۱۱)$$

loss نمونه:

$$\mathcal{L}_{\text{JEPA}} = \frac{1}{|\mathcal{T}|} \sum_{i \in \mathcal{T}} \|\hat{y}_i - \text{sg}(\mathbf{y}_i^{\text{target}})\|_1. \quad (۷۴.۱۱)$$

هدف این است که مدل جزئیات غیرقابل پیش‌بینی پیکسل را نادیده بگیرد و ساختار سطح بالاتر را پیش‌بینی کند.

۱۰۱۴.۱۱ چرا sampling بلوک مهم است؟

اگر target بسیار کوچک باشد، texture محلی کافی است؛ اگر context بسیار بزرگ و target نزدیک باشد، copy یا interpolation ممکن است؛ اگر context خیلی کوچک باشد، هدف غیرقابل پیش‌بینی است. در I-JEPA target نسبتاً بزرگ و context فضایی توزیع‌شده، مدل را به semantics سوق می‌دهد. بنابراین masking تنها حذف داده نیست، طراحی سطح دشواری و نوع اطلاعات است.

۲۰۱۴.۱۱ JEPA در برابر MAE

ویژگی	MAE	I-JEPA
فضای هدف	پیکسل/وصله	embedding معلم

decoder/predictor	decoder بازسازی	predictor نهفته
جزئیات تصادفی	باید تا حدی بازسازی شود	می تواند نادیده گرفته شود
collapse	هدف پیکسل متغیر	EMA encoder target + طراحی predictor
حساسیت کلیدی	ratio mask و target-normal sampling	context/target و کیفیت target
کاربرد طبیعی	پیش آموزش عمومی و بازسازی	نمایش معنایی و پیش بینی

۱۵.۱۱ مدل های بنیادی خودنظارتی

۱۰۱۵.۱۱ DINOv۲: روش، داده و مهندسی مقیاس

DINOv2 تنها یک loss جدید نیست؛ ترکیبی از، patch masked self-distillation، objective، داده گزینی، scale at training و distillation به مدل های کوچک تر است. این نکته برای دانشجو مهم است: در مقیاس بنیادین، کیفیت داده، deduplication، schedule balancing و pipeline توزیع شده بخشی از الگوریتم اند.

برای ارزیابی ویژگی عمومی، باید وظایف جهانی و مترکم جدا شوند: classification، kNN/linear، robustness، correspondence depth، segmentation، retrieval، اما feature patch ضعیف یا برعکس داشته باشد.

۲۰۱۵.۱۱ token ها register

در ViT بزرگ، بعضی patch ها token ممکن است به نقش storage جهانی تبدیل و norm غیرعادی پیدا کنند. register های token آموختنی فضای جدا برای این نقش فراهم می کنند تا patch ها token مکانی تر بمانند. register خروجی مکانی مستقیم ندارند و در dense decoder باید جدا شوند.

۳۰۱۵.۱۱ DINOv۳ و مقیاس جدید

DINOv3 مسیر backbone foundation self-supervised را به مقیاس بزرگ تر ادامه می دهد و بر کیفیت feature، dense سازگاری وضوح و انتقال بدون fine-tuning تأکید دارد. recipe رسمی شامل مرحله پیش آموزش، پالایش ساختار نمایش و سازگاری وضوح بالا است. در تدریس باید میان سه سطح تمایز گذاشت:

۱. objective بنیادی، teacher-student

۲. recipe مقیاس و داده؛

۳. distillation. و adaptation post-hoc

بازسازی کامل چنین مدل‌هایی در کلاس ممکن نیست، اما استخراج ویژگی، linear، corre-dense probe، spondence و تحلیل resolution قابل انجام است.

۴.۱۵.۱۱ - EVA-۰۲

EVA-02 از modeling image masked برای بازسازی ویژگی‌های یک teacher هم‌تراز با زبان استفاده می‌کند. این روش مرز میان self-supervised بصری و distillation از مدل vision-language را کمرنگ می‌کند. target قوی می‌تواند semantics را منتقل کند، اما bias و محدودیت teacher نیز منتقل می‌شود.

۱۶.۱۱ پیش‌آموزش تصویر - متن: SigLIP و SigLIP CLIP

گرچه نظارت طبیعی متن کاملاً «بدون برچسب» نیست، این خانواده برای فهم مدل‌های بنیادی ضروری است. یک encoder تصویر f و encoder متن g embedding نرمال‌شده می‌سازند. در CLIP، loss کنتراستی دوطرفه روی ماتریس شباهت batch اعمال می‌شود:

$$S_{ij} = \frac{f(x_i)^T g(t_j)}{\tau}. \quad (۷۵.۱۱)$$

$$\mathcal{L}_{\text{CLIP}} = \frac{1}{2} [\text{CE}(S, \text{diag}) + \text{CE}(S^T, \text{diag})]. \quad (۷۶.۱۱)$$

متن فضای مفهومی باز و classification zero-shot را فراهم می‌کند، اما caption ممکن است مکان، شمارش یا جزئیات ریز را پوشش ندهد.

۱.۱۶.۱۱ SigLIP

SigLIP به جای softmax سراسری، sigmoid loss زوجی استفاده می‌کند. اگر $y_{ij} = +1$ برای زوج صحیح و -1 برای زوج نادرست:

$$\mathcal{L}_{\text{sig}} = \sum_{i,j} \log(1 + \exp[-y_{ij}(s_{ij} + b)]). \quad (۷۷.۱۱)$$

این objective وابستگی normalization به کل batch را کاهش می‌دهد و batch scaling را تغییر می‌دهد.

۲.۱۶.۱۱ SigLIP

SigLIP 2 loss زبان - تصویر را با masked self-distillation، pretraining، captioning-based و prediction و curation data ترکیب می‌کند. نسخه‌های چندوضوح و حفظ نسبت تصویر نشان می‌دهند

توکن سازی بومی و self-supervision اکنون بخشی از recipe مدل چندوجهی اند. ویژگی مترکم قوی برای localization نیز اهمیت دارد.

پل به دهه آینده

مرز میان «مدل بصری خودنظارتی»، «مدل تصویر - متن» و «vision encoder مدل چندوجهی» در حال محو شدن است. فصل آینده باید مدل مولد و model foundation را بر اساس objective، داده و interface تحلیل کند، نه صرفاً نام محصول.

۱۷.۱۱ انتقال نمایش به وظایف پایین دست

۱۰۱۷.۱۱ kNN و probe linear

در kNN، frozen backbone است و شباهت ها embedding تصمیم را می سازد. اگر ویژگی ها نرمال باشند، cosine و ضرب داخلی هم ارزند. kNN: weighted

$$\text{score}(c | z) = \sum_{i \in \mathcal{N}_k(z)} \exp(s_i / \tau_{knn}) [y_i = c]. \quad (۷۸.۱۱)$$

Linear probe یک لایه خطی روی ویژگی frozen آموزش می دهد. این معیار separability خطی را می سنجد، نه ظرفیت کامل. fine-tuning.

۲۰۱۷.۱۱ fine-tuning کامل و لایه ای

در fine-tuning کامل، نرخ یادگیری backbone معمولاً کمتر از head است. learning-rate layer-wise decay:

$$\eta_l = \eta_{\text{top}} \alpha^{L-l}, \quad 0 < \alpha < 1 \quad (۷۹.۱۱)$$

لایه های پایین را محافظه کارتر تغییر می دهد. decay weight نباید روی bias و پارامترهای normalization بدون تصمیم آگاهانه اعمال شود. embedding positional و projection patch برای وضوح جدید نیازمند adaptation هستند.

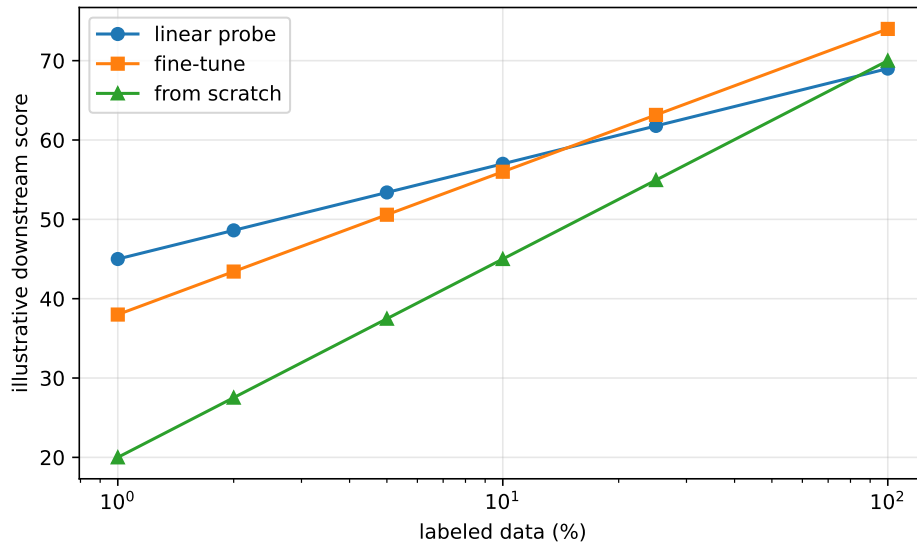
۳۰۱۷.۱۱ LoRA adapter و tuning prompt

برای ماتریس وزن $W \in \mathbb{R}^{d_o \times d_i}$ ، LoRA تغییر کم رتبه:

$$\Delta W = AB, \quad A \in \mathbb{R}^{d_o \times r}, \quad B \in \mathbb{R}^{r \times d_i}, \quad r \ll d \quad (۸۰.۱۱)$$

می‌آموزد. adapter یک residual bottleneck به بلوک می‌افزاید و prompt visual توکن‌های آموختنی را به دنباله اضافه می‌کند. کاهش پارامتر trainable لزوماً کاهش حافظه activation یا latency نیست؛ benchmark باید end-to-end باشد.

۴۰۱۷۰۱۱ داده کم برچسب



شکل ۲۱۰۱۱: منحنی آموزشی نمونه برای مقایسه linear probe، fine-tuning و آموزش از صفر در نسبت‌های مختلف برچسب. اعداد واقعی نیستند.

در داده بسیار کم، probe linear یا adapter می‌تواند overfitting را کم کند. با افزایش برچسب و domain، fine-tuning shift، کامل سودمندتر می‌شود. انتخاب باید با منحنی، label-efficiency چند seed و فاصله اطمینان انجام شود.

۱۸۰۱۱ انتقال متراکم: از token patch تا نقشه ویژگی

برای ViT با $H_p \times W_p$ grid، patch‌های token لایه l را می‌توان به

$$\mathbf{F}_l \in \mathbb{R}^{H_p \times W_p \times D_l} \quad (۸۱۰۱۱)$$

بازشکل داد. decoder قطعه‌بندی یا عمق از یک یا چند لایه استفاده می‌کند. ViT ساده هرم طبیعی ندارد؛ راهکارها:

- projection چند لایه به مقیاس‌های مصنوعی؛
- backbone سلسله‌مراتبی مانند Swin/Hiera؛
- adapter کانولوشنی؛

• decoder، pyramid feature

• استفاده از patch کوچک یا وضوح بالاتر.

در dense، interpolation transfer، ویژگی و alignment مختصات باید با مرکز پیکسل، padding و stride سازگار باشد. خطای یک نیم پیکسل می تواند در مرزها مهم شود.

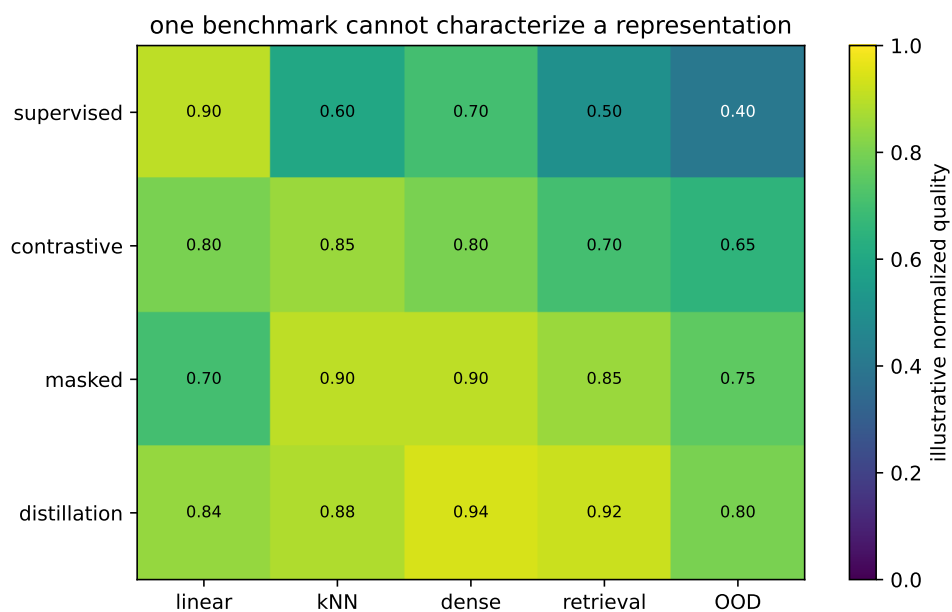
۱۰۱۸۰۱۱ correspondence و تطبیق با ویژگی خودنظارتی

برای دو تصویر، patch های feature نرمال F^A, F^B ماتریس شباهت

$$S = F^A (F^B)^T \quad (۸۲۰۱۱)$$

می سازند. neighbor nearest اولیه باید با mutual، check هندسه، scale و occlusion پالایش شود. ویژگی dense DINO برای این کار مفید است، اما resolution و patch size حد دقت مکانی را تعیین می کنند.

۱۹۰۱۱ ارزیابی یک نمایش: یک عدد کافی نیست



شکل ۲۲۰۱۱: یک objective می تواند در معیارهای جهانی و متریک رفتار متفاوت داشته باشد؛ جدول فقط مفهوم چندمحوری بودن ارزیابی را نشان می دهد.

۱۰۱۹۰۱۱ پروتکل های اصلی

۱. kNN: بدون آموزش head پیچیده؛ حساس به normalization و k ؛

۲. **probe: linear** separability خطی؛ حساس به epoch و optimizer
۳. **fine-tuning:** ظرفیت کامل انتقال؛ حساس به recipe
۴. **few-shot:** منحنی عملکرد بر حسب برچسب؛
۵. **retrieval:** mAP Recall@K و deduplication
۶. **transfer: dense** mIoU، error depth AP، correspondence
۷. **robustness:** corruption، OOD shift و adversarial
۸. **calibration:** Brier NLL و ECE
۹. کارایی: زمان پیش‌آموزش، انرژی، حافظه، latency و throughput.

۲۰۱۹۰۱۱ پروتکل probe linear عادلانه

backbone باید در حالت eval باشد؛ train augmentation و eval جدا؛ normalization feature مشخص؛ head و optimizer برای همه روش‌های یکسان؛ و اگر ویژگی چند لایه یا pooling انتخاب می‌شود، انتخاب روی validation انجام شود. گزارش بهترین تنظیم برای یک مدل و تنظیم پیش‌فرض برای مدل دیگر مقایسه را محدود می‌کند.

۳۰۱۹۰۱۱ ارزیابی بدون نشت

در داده بیمارمحور، ویدئویی یا مکانی، split باید در سطح هویت مستقل باشد. دو crop از یک تصویر یا فریم‌های یک ویدئو در train/test نشت ایجاد می‌کنند. در pretraining self-supervised نیز overlap داده پیش‌آموزش با benchmark باید مستند شود، به‌ویژه برای مدل‌های وب‌مقیاس.

۴۰۱۹۰۱۱ عدم قطعیت و OOD

feature قوی به معنی اعتماد درست نیست. head خطی می‌تواند overconfident باشد. علاوه بر accuracy، performance روی domain، Mahalanobis score، energy entropy، calibration distance و performance روی domain shift سنجیده شود. مدل بنیادی ممکن است در داده طبیعی عالی و در modality تخصصی فاقد اطلاعات رنگ/هندسه مناسب باشد.

۲۰۰۱۱ گنجینه‌ها: از kernel تا recipe خودنظارتی

runtime	CUDA / ROCm / CPU / ONNX Runtime / TensorRT
kernels	PyTorch SDPA / FlashAttention / xFormers
models	timmm / torchvision / Hugging Face Transformers
SSL recipes	Lightly / solo-learn / MMSelfSup / official DINO-MAE repos
experiments	Lightning / Accelerate / MMEEngine / W&B or MLflow

شکل ۲۳.۱۱: لایه‌های ابزار. انتخاب library باید بر اساس سطح انتزاع و نیاز بازتولیدپذیری باشد.

PyTorch ۱.۲۰.۱۱

برای آموزش از صفر، اجزای اصلی عبارت‌اند از:

- Conv2d یا reshape برای patch، embedding
- LayerNorm و Linear
- scaled_dot_product_attention برای kernel بهینه
- DistributedDataParallel برای چند GPU
- autocast و GradScaler برای mixed، precision
- torch.compile در صورت سازگاری graph و backend.

در SSL، all-gather، synchronized embedding، queue BatchNorm/centering و EMA نیازمند مدیریت توزیع شده‌اند.

۲۰۲۰.۱۱ timm و torchvision

torchvision مدل‌ها، transforms و extraction feature رسمی PyTorch را فراهم می‌کند. timm مجموعه گسترده‌تری از، backboneها، وزن‌ها، ها، optimizer augmentation و script training دارد. هنگام بارگذاری وزن:

۱. config preprocessing را از خود وزن بخوانید؛

۲. mean/std، percentage crop interpolation و resolution را ثبت کنید؛

۳. `num_classes=0` یا `features_only=True` را آگاهانه انتخاب کنید؛

۴. نام `checkpoint` و `revision` را ثابت کنید.

۳.۲۰.۱۱ Transformers Face Hugging

این کتابخانه مدل‌های ViT، ViTMAE، BEiT، DINOv2/v3، I-JEPA، Swin و بسیاری en-های vision coder را با `config` و `processor` یکپارچه ارائه می‌کند. تفاوت `AutoImageProcessor` با `transform` دستی باید روشن باشد؛ دو `preprocessing` متفاوت می‌توانند `performance` را تغییر دهند.

۴.۲۰.۱۱ MMSelfSup و solo-learn Lightly

این ابزارها برای SSL recipe را آماده می‌کنند:

- `Lightly`: `transform loss` و مثال‌های روش‌های کنتراستی و `non-contrastive`
- `solo-learn`: مجموعه پژوهشی روش‌های SSL برای مقایسه؛
- `MMSelfSup`: `training config-based` در اکوسیستم `OpenMMLab`
- مخازن رسمی `MAE` و `DINOv2/v3`: مرجع جزئیات واقعی `recipe`.

استفاده از کتابخانه، مسئولیت فهم `objective` و `config` را حذف نمی‌کند. نسخه، `commit` و تغییرات محلی باید ثبت شوند.

۵.۲۰.۱۱ WebDataset و داده وب مقیاس

برای میلیون‌ها تصویر، فایل‌های کوچک جداگانه `metadata bottleneck` ایجاد می‌کنند. `shard`، `tar`، `deduplication`، `resampling streaming` و `prefetch` راهکار رایج‌اند. اما `shuffle` تقریبی، خرابی نمونه، `deduplication` و `reproducibility` دشوارتر می‌شود. شمار نمونه مؤثر هر `epoch` باید `log` شود.

کتابخانه و ابزار: مخازن رسمی به‌عنوان مشخصات اجرایی

مقاله `objective` را توضیح می‌دهد، اما جزئیاتی مانند `warm-up` دما، `teacher mask`، `schedule`، `weight momentum`، `clipping gradient decay` و `augmentation` اغلب در `config` رسمی دقیق‌ترند. برای بازسازی، `paper` و `repository` باید با هم خوانده شوند؛ کپی `config` بدون فهم سخت‌افزار و `batch` نیز کافی نیست.

۲۱.۱۱ مهندسی آموزش در مقیاس

۱۰.۲۱.۱۱ batch مؤثر و accumulation gradient

اگر batch هر GPU برابر B_g ، تعداد GPU برابر G و تعداد گام accumulation برابر A باشد:

$$B_{\text{eff}} = B_g G A. \quad (۸۳.۱۱)$$

اما accumulation دقیقاً معادل batch بزرگ نیست اگر BatchNorm، update teacher queue یا schedule بر حسب step به روز شوند. در SSL کنتراستی، تعداد های negative هم زمان ممکن است تنها $B_g G$ باشد، نه B_{eff} ، مگر embedding چند microbatch ذخیره شود.

۲۰.۲۱.۱۱ EMA در آموزش توزیع شده

تمام rankها باید دانش آموز یکسان داشته باشند؛ سپس teacher به صورت محلی با EMA به روزرسانی می شود. اگر operation stochastic یا state optimizer ناهمگام باشد، ها teacher واگرا می شوند. center و optimizer، center، checkpoint باید هر دو شبکه، optimizer، center، progress schedule و scaler را ذخیره کند.

۳۰.۲۱.۱۱ precision mixed

در ۱۶FP، logits softmax و variance کوچک می توانند overflow/underflow کنند. ۱۶BF دامنه exponent بزرگ تری دارد و برای transformer معمولاً پایدارتر است، اگر سخت افزار پشتیبانی کند. nor-malization و reduction loss ممکن است در ۳۲FP اجرا شوند. نوع precision بخشی از recipe است و باید گزارش شود.

۴۰.۲۱.۱۱ decay weight و پارامترهای مستثنا

در ViT معمولاً decay weight روی ماتریس های وزن اعمال می شود، اما LayerNorm bias و گاهی positional/class/register tokenها مستثنا هستند. اگر مجموعه پارامترها به اشتباه ساخته شود، تغییر کوچک config می تواند نتیجه را جابه جا کند. کد باید نام و تعداد پارامتر هر گروه optimizer را چاپ کند.

۵۰.۲۱.۱۱ های schedule متداول

نرخ یادگیری با batch مقیاس داده می شود، warm-up اولیه و decay cosine رایج اند:

$$\eta(t) = \eta_{\min} + \frac{1}{4}(\eta_{\max} - \eta_{\min}) \left[1 + \cos \left(\pi \frac{t - t_w}{T - t_w} \right) \right]. \quad (۸۴.۱۱)$$

Teacher، decay weight momentum و دمای teacher نیز ممکن است schedule مستقل داشته باشند. نمایش فقط منحنی loss بدون نمایش ها schedule برای عیب یابی کافی نیست.

۶.۲۱.۱۱ checkpoint و بازتولیدپذیری

یک checkpoint پژوهشی باید شامل موارد زیر باشد:

- وزن student و teacher
- scheduler optimizer و AMP، scaler
- state، sampler و step global epoch
- center/prototype/queue
- RNG های state، Python، NumPy CPU و CUDA
- resolved config و commit کد
- manifest داده یا shard، list

Resume از checkpoint باید با آزمون کوتاه بررسی شود که loss و schedule از همان نقطه ادامه می‌یابند.

۲۲.۱۱ طراحی آزمایش‌های دقیق

۱.۲۲.۱۱ آزمایش ۱: اندازه وصله

مدل، داده و بودجه آموزش ثابت؛ $P \in \{8, 16, 32\}$. گزارش:

۱. تعداد توکن و memory، peak
۲. throughput آموزش و inference
۳. fine-tuning و probe linear
۴. عملکرد شیء کوچک/بافت ریز
۵. sensitivity به تغییر وضوح.

اگر batch به دلیل حافظه تغییر می‌کند، optimizer و schedule باید برای مقایسه تنظیم یا این confound صریح گزارش شود.

۲.۲۲.۱۱ آزمایش ۲: کدگذاری مکان

مقایسه embedding مطلق، bias relative و RoPE D۲ روی آموزش ۲۲۴ و ارزیابی ۲۲۴، ۳۸۴ و نسبت تصویر غیرمربع. معیارها: accuracy، localization dense calibration و artefact مکانی. فقط interpolation موفق tensor به معنی تعمیم هندسی نیست.

۳.۲۲.۱۱ آزمایش ۳: دمای InfoNCE

norm گزادیان، rank مؤثر و probe linear را رسم کنید. بهترین loss ممکن است بهترین انتقال نباشد. $\tau \in \{0.05, 0.1, 0.2, 0.5\}$ با batch و augmentation ثابت. علاوه بر similarity loss مثبت/منفی،

۴.۲۲.۱۱ آزمایش ۴: نسبت mask

برای MAE نسبت $\{0.5, 0.65, 0.75, 0.9\}$ ، زمان هر epoch، کیفیت probe linear reconstruction، transfer dense را گزارش کنید. capacity decoder ثابت باشد. ratio mask بالا encoder را ارزان می کند، بنابراین مقایسه باید هم epoch ثابت و هم compute ثابت داشته باشد.

۵.۲۲.۱۱ آزمایش ۵: فضای هدف

با backbone و داده یکسان، هدف پیکسل، پیکسل نرمال شده، teacher feature و token گسسته مقایسه شود. معیارهای جهانی و متراکم جدا گزارش شوند. این آزمایش نشان می دهد objective چه اطلاعاتی را ترجیح می دهد.

۶.۲۲.۱۱ آزمایش ۶: augmentation دامنه ای

برای داده پزشکی/صنعتی، هر augmentation ابتدا از نظر فیزیکی توجیه و سپس ablation شود. جدول پیشنهادی:

تبدیل	ناوردایی مطلوب	خطر حذف سیگنال	آزمون تشخیصی
crop	مکان و کادر	حذف ضایعه/عیب	ROI با overlap
jitter color	نورگیری	تغییر نشانگر رنگی	تحلیل کانال
blur	کیفیت حسگر	حذف بافت ریز	عملکرد بر حسب فرکانس
rotation	جهت دوربین	جهت تشخیصی	گروه بندی زاویه
flip	تقارن	چپ/راست معنایی	ارزیابی سمت

۷.۲۲.۱۱ آزمایش ۷: پیش آموزش درون دامنه و برون دامنه

سه random، initialization: مدل عمومی و SSL روی داده بدون برچسب دامنه. برای چند نسبت برچسب، منحنی عملکرد و فاصله اطمینان گزارش شود. اگر داده دامنه کم و یکنواخت است، SSL درون دامنه ممکن است overfit یا collapse کند؛ ترکیب داده عمومی و تخصصی بررسی شود.

۸.۲۲.۱۱ آزمایش ۸: pruning/merging token

برای های budget مختلف، علاوه بر top-۱، معیار dense و worst-group را گزارش کنید. latency واقعی روی سخت افزار هدف و های batch ۱ و بزرگ سنجیده شود. کاهش FLOPs ممکن است به scatter/gather نامنظم و کاهش اندک سرعت منجر شود.

۹.۲۲.۱۱ آزمایش ۹: features dense frozen

یک decoder خطی یا بسیار سبک روی patch های feature لایه های مختلف آموزش دهید. این آزمون نشان می دهد semantics محلی در کدام عمق ظاهر می شود. resolution و interpolation را ثابت نگه دارید.

۱۰.۲۲.۱۱ آزمایش ۱۰: ممیزی فروپاشی

در یک اجرای عمداً ناپایدار، یکی از variance centering، stop-gradient term یا ها negative حذف شود. spectrum loss و rank مؤثر ثبت شود. این آزمایش آموزشی نشان می دهد چرا هر سازوکار وجود دارد.

خواندن مقاله و استاندارد: حداقل جدول گزارش پیش آموزش

مؤلفه	آنچه باید گزارش شود
داده	تعداد نمونه، dedup، split، license، sampling filtering
ورودی	normalization، patch، policy، aspect resolution، کانال
augmentation	ترتیب، احتمال و بازه پارامتر هر تبدیل
مدل	registers position، ratio، MLP heads، width، depth
objective	loss کامل، وزن جمله ها، target و mask temperature
teacher	frequency update centering، schedule، momentum
بهینه سازی	precision clipping، decay، warm-up، lr، optimizer
توزیع	statistics sync all-gather، accumulation، batch، GPU
هزینه	memory peak، energy GPU-hours در صورت امکان
ارزیابی	variance و robustness dense، fine-tune، linear، kNN، seed

۲۳.۱۱ عیب‌یابی سیستماتیک

آزمون تشخیصی: loss خودنظارتی کم نمی‌شود

۱. mapping positive و batch ordering را بررسی کنید؛
۲. embedding normalization و دما را چاپ کنید؛
۳. ها augmentation را تصویری ذخیره کنید؛
۴. all-gather و offset label را در چند GPU بررسی کنید؛
۵. rate learning و warm-up را نسبت به batch بازبینی کنید؛
۶. backbone norm gradient و projector را جدا جدا log کنید.

آزمون تشخیصی: loss کم است اما probe linear ضعیف

ممکن است pretext بیش از حد آسان، augmentation نامناسب، collapse، space projection، nearest-neighbor، shortcut رنگ/مرز، یا داده تکراری باشد. ویژگی قبل و بعد، projector، spectrum، transfer و لایه‌های مختلف را مقایسه کنید.

آزمون تشخیصی: MAE تصویر تار بازسازی می‌کند

MSE میانگین شرطی را ترجیح می‌دهد؛ reconstruction تار لزوماً نمایش بد نیست. target-norm، capacity decoder malization، mask را بررسی کنید، اما مدل را بر اساس هدف پایین‌دست قضاوت کنید، نه زیبایی تصویر بازسازی‌شده.

آزمون تشخیصی: DINO collapse یا entropy نامناسب

teacher، temperature، center warm-up، update، momentum، head normalization و precision را بررسی کنید. batch بسیار کوچک یا داده کم تنوع می‌تواند statistics را بی‌ثبات کند. خروجی معلم و دانش‌آموز را جدا جدا log کنید.

آزمون تشخیصی: مدل در وضوح جدید افت می‌کند

positional، patch interpolation، policy crop grid و distribution scale آموزش را بررسی کنید. افزایش resolution بدون fine-tuning ممکن است توزیع توکن و فاصله‌های نسبی را تغییر دهد. training native-resolution یا fine-tuning multi-resolution را آزمایش کنید.

۲۴.۱۱ مرزهای پژوهش در ۲۰۲۵-۲۰۲۶

۱۰.۲۴.۱۱ ویژگی متراکم در مقیاس بنیادین

مدلهایی مانند DINOv3 نشان می‌دهند هدف فقط embedding طبقه‌بندی نیست؛ backbone باید patch های feature عمومی برای وظایف متراکم تولید کند. چالش‌ها شامل حفظ جزئیات در مقیاس بزرگ، جلوگیری از adaptation artefact و وضوح بالا و distillation به مدل قابل استفاده است.

۲۰.۲۴.۱۱ ترکیب چند هدف

SigLIP 2 نمونه‌ای از recipe ترکیبی است: زبان - تصویر، prediction masked self-distillation و curation. data سؤال پژوهشی آینده کمتر «کدام loss واحد برنده است؟» و بیشتر «چه ترکیب هدف برای کدام نوع انتقال و داده بهینه است؟» خواهد بود. training multi-objective به توازن gradient و زمان‌بندی نیاز دارد.

۳۰.۲۴.۱۱ وضوح و نسبت تصویر بومی

مدل‌های native-resolution از resize اجباری فاصله می‌گیرند. چالش عملی: packing کارآمد، posi-tional encoding سازگار، batch نامنظم، augmentation هندسی و benchmark منصفانه. برای اسناد، پزشکی و سنجش‌ازدور، حفظ نسبت تصویر می‌تواند مهم‌تر از افزایش صرف resolution باشد.

۴۰.۲۴.۱۱ توکن پویا و بودجه تطبیقی

مدل می‌تواند برای تصویر ساده توکن کمتر و برای صحنه پیچیده توکن بیشتر استفاده کند. objective باید accuracy و compute را هم‌زمان بهینه کند:

$$\min_{\theta} \mathbb{E}[\mathcal{L}_{\text{task}}(x; \theta)] + \lambda \mathbb{E}[\mathcal{C}(x; \theta)]. \quad (۸۵.۱۱)$$

اما compute باید latency واقعی و محدودیت سخت‌افزار را نمایندگی کند، نه فقط differentiable FLOPs surrogate.

۵۰.۲۴.۱۱ خودنظارتی بدون augmentation دست‌ساز شدید

JEPA و objectives predictive تلاش می‌کنند semantics را از پیش‌بینی context/target استخراج کنند و وابستگی به های invariance دستی را کم کنند. با این حال، mask sampling خود نوعی bias inductive است. پژوهش آینده باید، masking augmentation و داده‌گزینی را مشترکاً یاد بگیرد یا دست کم حساسیت را اندازه‌گیری کند.

۶.۲۴.۱۱ هم ترازی متن و حفظ هندسه

ویژگی تصویر - متن ممکن است semantics جهانی قوی اما localization ضعیف داشته باشد. ترکیب masked/self-distillation و objective dense می تواند این شکاف را کم کند. ارزیابی باید هم zero-retrieval shot و هم correspondence/segmentation را شامل شود.

۷.۲۴.۱۱ داده گزینی و حاکمیت داده

در مدل وب مقیاس، deduplication، کیفیت، تنوع، حریم خصوصی، license و bias بخشی از روشن اند. افزایش تعداد تصویر بدون مستندسازی منبع و، filtering ادعای علمی را ناقص می کند. مدل عمومی ممکن است داده benchmark یا محتوای حساس را دیده باشد؛ ارزیابی contamination ضروری است.

۸.۲۴.۱۱ ویژگی ها به عنوان زیرساخت

های frozen backbone می توانند به زیرساختی مانند SIFT مدرن تبدیل شوند: استخراج، feature، indexing، clustering matching و lightweight. adaptation برای این نقش، API پایدار، مجوز، هزینه، inference، نسخه checkpoint و رفتار در shfit domain اهمیت برابر با benchmark دارند.

۲۵.۱۱ الگوریتم های مرجع فصل

الگوریتم ۱: ViT پیش هنجار شده

۱. تصویر را با P kernel/stride به توکن تبدیل کنید؛
۲. توکن CLS و position را اضافه کنید؛
۳. برای $l = 1$ تا L : $X \leftarrow X + \text{MHA}(\text{LN}(X))$ ؛
۴. سپس $X \leftarrow X + \text{MLP}(\text{LN}(X))$ ؛
۵. خروجی CLS یا میانگین ها token را به head بدهید.

الگوریتم ۲: SimCLR توزیع شده

۱. از هر تصویر دو نمای مستقل بسازید؛
۲. encoder و projector را روی هر دو نما اجرا کنید؛
۳. embedding ها را روی همه ها rank با gather gradient-aware جمع کنید؛
۴. positive ها index را بر اساس rank و ordering بسازید؛

۵. NT-Xent دوطرفه را محاسبه کنید؛

۶. scheduler optimizer و logger را به روزرسانی کنید.

الگوریتم ۳: MoCo با صف

۱. query را با online encoder و key را بدون gradient با momentum encoder بسازید؛

۲. logit positive را از $q \cdot k^+$ و ها negative را از صف بسازید؛

۳. cross-entropy را محاسبه و query encoder را به روزرسانی کنید؛

۴. key encoder را با EMA به روزرسانی کنید؛

۵. های key جدید را enqueue و قدیمی ها را dequeue کنید.

الگوریتم ۴: DINO چندبرشی

۱. دو crop سراسری و چند crop محلی تولید کنید؛

۲. دانش آموز همه ها crop و معلم فقط های crop سراسری را پردازش کند؛

۳. خروجی معلم را center و sharpen کنید؛

۴. cross-entropy همه زوج های نمای متفاوت را میانگین بگیرید؛

۵. دانش آموز را با gradient و معلم را با EMA به روزرسانی کنید؛

۶. center را با میانگین متحرک توزیع شده به روزرسانی کنید.

الگوریتم ۵: MAE

۱. patchify و embedding مکان را بسازید؛

۲. permutation تصادفی و visible subset را انتخاب کنید؛

۳. فقط ها visible را به encoder بدهید؛

۴. ها latent و mask ها token را با index restoration مرتب کنید؛

۵. decoder را اجرا و فقط روی های patch masked loss بگیرید.

الگوریتم ۶: VICReg

۱. دو نما را encode و project کنید؛
۲. MSE زوج‌ها را برای invariance محاسبه کنید؛
۳. std هر بعد را محاسبه و کمبود نسبت به γ را جریمه کنید؛
۴. centered covariance هر شاخه و خارج قطر آن را جریمه کنید؛
۵. سه جمله را با وزن‌های ثبت‌شده جمع کنید.

الگوریتم ۷: I-JEPA

۱. target های block بزرگ و mask context توزیع‌شده نمونه‌گیری کنید؛
۲. encoder context فقط context را پردازش کند؛
۳. EMA encoder target نمایش target ها را بدون gradient بسازد؛
۴. predictor با context و embedding مکان، embedding target target را پیش‌بینی کند؛
۵. loss نهفته را محاسبه و student/predictor را به‌روزرسانی کنید.

الگوریتم ۸: probe linear استاندارد

۱. backbone و checkpoint preprocessing را ثابت کنید؛
۲. train/validation/test feature را بدون augmentation تصادفی ارزیابی استخراج کنید؛
۳. فقط head خطی را با tuning validation آموزش دهید؛
۴. mean/std چند seed و calibration را گزارش کنید؛
۵. feature، pooling layer و normalization را مستند کنید.

۲۶.۱۱ کارگاه یکپارچه: از صفر تا مدل آماده

۱۰.۲۶.۱۱ مسیر A: فهم از صفر

۱. patchify.py را اجرا و inverse را آزمون کنید؛
۲. minimal_vit.py را با های patch ۴ و ۸ مقایسه کنید؛
۳. تعداد پارامتر و shape token هر مرحله را چاپ کنید؛

۴. weights attention یا خروجی هاhead را برای ورودی مصنوعی ذخیره کنید؛
۵. norm gradient هر بلوک را بررسی کنید.

۲۰۲۶۰۱۱ مسیر: InfoNCE B و collapse

۱. info_nce.py را با دماهای مختلف اجرا کنید؛
۲. هاpositive را عمداً shuffle کنید و loss را مقایسه کنید؛
۳. embedding ثابت بسازید و spectrum را اندازه بگیرید؛
۴. head projection را حذف و probe linear را مقایسه کنید.

۳۰۲۶۰۱۱ مسیر: MAE C

۱. mask ratio را تغییر دهید؛
۲. loss را فقط masked و سپس همه هاpatch محاسبه و تفاوت را توضیح دهید؛
۳. normalization target را اضافه کنید؛
۴. depth decoder را تغییر دهید و هزینه/انتقال را بسنجید.

۴۰۲۶۰۱۱ مسیر: D مدل آماده

با timm یا Transformers یک مدل ViT/DINO/MAE را بدون head بارگذاری کنید، prepro-cessing رسمی را اعمال و feature استخراج کنید. در محیط بدون اینترنت، checkpoint باید از قبل در cache یا مسیر محلی باشد.

```

1  """Optional online example. It does not run during the offline smoke test."
   ""
2  from __future__ import annotations
3  import torch
4
5
6  def main():
7      try:
8          import timm
9      except ImportError as exc:
10         raise SystemExit('Install timm first: pip install timm') from exc
11     model = timm.create_model('vit_small_patch16_224', pretrained=False,
12                               num_classes=0)
13     model.eval()
14     x=torch.randn(1,3,224,224)
15     with torch.no_grad(): y=model(x)

```

```

15 print({'model':model.__class__.__name__, 'feature_shape':tuple(y.shape)
16       })
17 if __name__=='__main__': main()

```

قرارداد میان ریاضیات و کد

کدهای این فصل دو دسته‌اند: کدهای آموزشی که بدون دانلود و روی CPU test smoke می‌شوند؛ و کدهای اتصال به کتابخانه/وزن که ممکن است به نصب بسته و checkpoint نیاز داشته باشند. هیچ خروجی مصنوعی به عنوان نتیجه benchmark واقعی معرفی نمی‌شود.

۲۷.۱۱ تمرین‌های نظری و اثباتی

۱. نشان دهید projection وصله با کانولوشن $Pkernel=stride$ هم‌ارز است و mapping وزن بین دو نمایش را بنویسید.
۲. برای تصویر ۶۴۰×۳۸۴ و های patch ۳۲، ۱۶، ۸، تعداد توکن و اندازه ماتریس attention را محاسبه کنید.
۳. رابطه (۸.۱۱) را برای تصویر مربعی استخراج و اثر دوبرابر کردن وضوح را تحلیل کنید.
۴. اگر $q_r, k_r \sim \mathcal{N}(0, \sigma^2)$ مستقل باشند، میانگین و واریانس $q^T k$ را محاسبه کنید.
۵. ژاکوبین softmax را از تعریف مشتق کنید و نشان دهید هر سطر/ستون چه جمعی دارد.
۶. ثابت کنید self-attention بدون position نسبت به جایگشت هم‌ورداست.
۷. تعداد پارامتر و MAC یک بلوک ViT با $D = ۷۶۸$ ، $r = ۴$ ، $N = ۱۹۷$ را تقریب بزنید.
۸. هزینه attention پنجره‌ای ۷×۷ را با attention global برای ۵۶×۵۶ مقایسه کنید.
۹. نشان دهید InfoNCE cross-entropy مشتق رابطه (۴۸.۱۱) را دارد.
۱۰. نقش embedding normalization را در جلوگیری از افزایش بی‌کران norm برای کاهش loss کنترستی توضیح دهید.
۱۱. یک مثال negative false بسازید و جهت gradient آن را تحلیل کنید.
۱۲. کران اطلاعات متقابل InfoNCE را با فرض sampling صحیح بیان و محدودیت آن را توضیح دهید.
۱۳. برای EMA، وزن contribution پارامتر دانش آموز k گام قبل را استخراج کنید.
۱۴. Twins Barlow loss را برای $C = I$ و برای $C = ۱۱^T$ محاسبه کنید.
۱۵. نشان دهید اگر همه هالهای embedding ثابت باشند، جمله variance در VICReg مثبت می‌شود.
۱۶. rank مؤثر یک طیف یکنواخت و یک طیف تک‌بعدی را محاسبه کنید.

۱۷. برای MAE با نسبت mask r_m ، کاهش تقریبی هزینه encoder attention را نسبت به encoder کامل استخراج کنید.
۱۸. توضیح دهید چرا بازسازی پیکسل با MSE می‌تواند خروجی میانگین/تار ایجاد کند.
۱۹. MAE و I-JEPA را از دید متغیر تصادفی هدف و expectation conditional مقایسه کنید.
۲۰. نشان دهید RoPE چگونه ضرب داخلی را به اختلاف موقعیت وابسته می‌کند.
۲۱. اثر embedding interpolation مکان را در تغییر نسبت تصویر تحلیل کنید.
۲۲. یک mask attention بلوکی برای سه دنباله packed با طول‌های ۳، ۲، ۴ بنویسید.
۲۳. تفاوت کاهش FLOPs و کاهش latency را با مثال memory-bound توضیح دهید.
۲۴. یک معیار برای سنجش locality هر head پیشنهاد و محدودیت آن را بیان کنید.
۲۵. چرا probe linear قوی الزاماً fine-tuning قوی را تضمین نمی‌کند؟
۲۶. چه شرایطی باعث می‌شود rotation augmentation برای یک دامنه نامعتبر باشد؟
۲۷. برای SigLIP مشتق loss زوجی نسبت به logit را محاسبه کنید.
۲۸. تفاوت‌های negative درون queue batch و prototype را از دید bias/variance مقایسه کنید.
۲۹. چرا teacher بسیار کند می‌تواند target کهنه تولید کند و teacher سریع چه خطری دارد؟
۳۰. یک objective چندهدفه برای حفظ ویژگی جهانی و متراکم طراحی کنید.

۲۸.۱۱ تمرین‌های کدنویسی

۱. patchify را با unfold و d2Conv پیاده‌سازی و خروجی هر سه روش را مقایسه کنید.
۲. embedding positional دوبعدی سینوسی تولید و به TinyViT اضافه کنید.
۳. weights attention هر head را ذخیره و entropy آن را محاسبه کنید.
۴. یک attention window ساده و mask window shifted بنویسید.
۵. memory peak توجه global و window را برای طول‌های مختلف اندازه‌گیری کنید.
۶. InfoNCE را برای چند positive در هر anchor تعمیم دهید.
۷. نسخه MoCo با queue حلقوی و EMA encoder بسازید.
۸. Twins Barlow و VICReg را از صفر پیاده‌سازی و check gradient کنید.
۹. در TinyMAE normalization target و embedding positional را اضافه کنید.
۱۰. mask تصادفی و بلوکی را مقایسه و تصاویر mask را ذخیره کنید.

۱۱. یک teacher-student با centering و schedule دما بسازید.
۱۲. rank مؤثر و embedding spectrum را در طول آموزش log کنید.
۱۳. با timm ویژگی چند مدل را استخراج و CKA آن‌ها را محاسبه کنید.
۱۴. probe linear با regression logistic و با PyTorch head را مقایسه کنید.
۱۵. یک index retrieval با FAISS یا NumPy nearest-neighbor بسازید.
۱۶. patch های DINO/ViT feature را برای correspondence nearest-neighbor به کار ببرید.
۱۷. LoRA را برای های projection Q و V پیاده سازی کنید.
۱۸. packing sequence را با mask بلوکی در SDPA آزمایش کنید.
۱۹. توکن های مشابه را با الگوریتم merge greedy و کاهش طول دنباله پیاده سازی کنید.
۲۰. benchmark را با warm-up، median، p۹۵ و memory peak استاندارد کنید.

۲۹.۱۱ پروژه های پایان فصل

۱۰۲۹.۱۱ پروژه ۱: آزمایشگاه توکن سازی

یک چارچوب بسازید که patch ثابت، overlapping، FlexiViT-style stem، randomization patch و packing native-resolution را روی یک dataset مقایسه کند. خروجی باید شامل، dense accuracy، gallery failure و distance attention memory، latency، metric، باشد.

۲۰۲۹.۱۱ پروژه ۲: بازسازی پنج خانواده SSL

روی dataset متوسط و backbone یکسان، BYOL، SimCLR، یا SimSiam، یا MAE، VICReg و یک روش teacher-student را با budget برابر اجرا کنید. هدف، رتبه بندی مطلق نیست؛ تحلیل اینکه هر objective چه هندسه و profile transfer می سازد.

۳۰۲۹.۱۱ پروژه ۳: خودنظارتی دامنه تخصصی

برای داده پزشکی، صنعتی یا سنجش ازدور، های augmentation معتبر را با متخصص دامنه تعریف کنید. مدل عمومی، SSL درون دامنه و ترکیب دو مرحله ای را در label fractions مختلف مقایسه کنید. split در سطح بیمار/مکان/دستگاه ضروری است.

۴۰۲۹۰۱۱ پروژه ۴: frozen backbone با correspondence dense

ویژگی patch چند لایه را استخراج، matching متقابل و RANSAC هندسی را اضافه کنید و اثر -resolu- tion، patch size، layer و token register را بسنجید. با SIFT فصل نهم مقایسه کنید.

۵۰۲۹۰۱۱ پروژه ۵: مدل بودجه‌تطبیقی

یک ViT با pruning token یا merging بسازید. loss هزینه و task را ترکیب کنید و جبهه پارتوی accuracy-latency را روی سخت‌افزار واقعی گزارش دهید. تحلیل worst-case برای تصاویر پیچیده الزامی است.

۶۰۲۹۰۱۱ پروژه ۶: ممیزی مدل بنیادی

یک ۳/v۲DINOv، CLIP/SigLIP و MAE را روی وظایف global، dense، retrieval و OOD دامنه خود ارزیابی کنید. هیچ fine-tuning اولیه انجام ندهید؛ سپس adapter یا probe linear را اضافه و تغییر را تحلیل کنید.

۷۰۲۹۰۱۱ پروژه ۷: روش جدید ضد فروپاشی

یک تغییر نظری کوچک و قابل آزمون بر، DINO VICReg یا JEPA پیشنهاد دهید. باید، baseline، ablation سازوکار، diagnostic، spectrum، تحلیل حساسیت و case failure داشته باشد. صرف ترکیب نام دو loss بدون فرض روشن پذیرفتنی نیست.

۸۰۲۹۰۱۱ پروژه ۸: کتابخانه آموزشی فارسی

یک package کوچک شامل، patchify، ViT، loss، SSL، trainer و diagnostic dashboard بسازید. API، type، hint، test، docstring، مثال CPU و config بازتولیدپذیر لازم است.

۳۰۰۱۱ پرامپت پیشنهادی برای Codex

کارگاه کد و آزمایش: ساخت آزمایشگاه کامل فصل یازدهم

- 1 Act **as** a research software engineer **for** a graduate-level computer vision course.
- 2 Build a reproducible PyTorch project **for** Chapter 11: Vision Transformers **and**
- 3 Self-Supervised Learning.
- 4

```

5 Requirements:
6 1. Implement patchify/unpatchify, a minimal pre-LN ViT, 2D positional
7   global and window attention, and optional native-resolution
8   sequence packing.
9 2. Implement and unit-test NT-Xent/InfoNCE, MoCo queue + EMA encoder,
10  VICReg,
11  Barlow Twins, a small MAE, and a DINO-style teacher-student
12  objective.
13 3. Support CIFAR-10 and ImageFolder, but provide an offline synthetic
14  smoke mode.
15 4. Use dataclass/YAML configs. Record resolved config, seed, git
16  commit, package
17  versions, dataset manifest, parameter groups, and checkpoint state.
18 5. Add mixed precision, gradient accumulation, DDP-safe all-gather,
19  EMA, resume,
20  and TensorBoard/CSV logging. Never silently change the effective
21  batch size.
22 6. Log collapse diagnostics: per-dimension std, covariance spectrum,
23  effective
24  rank, positive/negative similarity, teacher/student entropy, and
25  grad norms.
26 7. Provide evaluation scripts for kNN, linear probing, full fine-
27  tuning, retrieval,
28  corruption robustness, calibration, latency, throughput, and peak
29  memory.
30 8. Add ablations for patch size, mask ratio, temperature,
31  augmentations,
32  positional encoding, token pruning, and feature layer.
33 9. Use pytest for shape, gradient, queue, mask restoration,
34  distributed index,
35  and checkpoint-resume tests.
36 10. Do not invent benchmark results. Clearly label synthetic outputs.
37  Fail with
38  informative errors when optional dependencies or datasets are
39  missing.
40
41 Deliver a clean directory tree, README in Persian and English,
42 commands for CPU
43 smoke tests and GPU experiments, and a manuscript-ready CSV/figure
44 export.

```

۳۱.۱۱ تقسیم پیشنهادی ویدئوهای ۱۵ دقیقه‌ای

تقسیم پیشنهادی ویدئوهای ۱۵ دقیقه‌ای

موضوع	بخش
چرا تصویر را توکن می‌کنیم؟ تفاوت unit در CNN و ViT	۱
patchify به عنوان بازآرایی تانسور	۲
هم‌ارزی projection patch و Conv۲d	۳
تعداد توکن و قانون هزینه P^{-4}	۴
اندازه وصله، فرکانس و آلیاسینگ	۵
توکن‌های هم‌پوشان و stem کانولوشنی	۶
توکن‌سازی سلسله‌مراتبی و merging patch	۷
TokenLearner و توکن محتوای محور	۸
DynamicViT و merging pruning،	۹
NaViT و بسته‌بندی وضوح بومی	۱۰
FlexiViT و یک مدل برای چند patch size	۱۱
توکن شیء، ناحیه و token گسسته	۱۲
Q، K و V از دید جبر خطی	۱۳
استخراج scaled dot-product attention	۱۴
چرا $1/\sqrt{d_k}$ ؟	۱۵
softmax، ژاکوبین و اشباع	۱۶
attention به عنوان regression kernel	۱۷
attention به عنوان گراف داده‌محور	۱۸
attention multi-head و ها projection	۱۹
بلوک، residual Pre-LN و MLP	۲۰
پیاده‌سازی TinyViT از صفر	۲۱
attention map چرا توضیح کامل نیست؟	۲۲
جایگشت و ضرورت اطلاعات مکان	۲۳
embedding مطلق و interpolation	۲۴
کد سینوسی دوبعدی	۲۵
bias position relative	۲۶
RoPE دوبعدی	۲۷
معماری پایه ViT و شمار پارامتر	۲۸
CLS token در برابر pooling mean	۲۹
DeiT و token distillation	۳۰

۳۱	attention window و Swin
۳۲	mask و window shifted و مرزی
۳۳	PVT و reduction spatial
۳۴	MViT و attention pooling
۳۵	MaxViT و attention multi-axis
۳۶	Hiera و رابطه معماری با MAE
۳۷	هزینه درجه دو و حافظه attention
۳۸	attention efficient خانواده‌های
۳۹	منطق FlashAttention
۴۰	SDPA PyTorch و benchmark صحیح
۴۱	تعریف یادگیری خودنظارتی
۴۲	augmentation به عنوان نظارت
۴۳	projector encoder، representation و
۴۴	InfoNCE و NT-Xent
۴۵	مشتق دما و logits
۴۶	alignment و uniformity
۴۷	اطلاعات متقابل و محدودیت تفسیر
۴۸	batch و recipe SimCLR:
۴۹	MoCo: صف و encoder تکرانه‌ای
۵۰	multi-positive و negative false
۵۱	SwAV و assignment prototype
۵۲	BYOL و predictor
۵۳	SimSiam و stop-gradient
۵۴	DINO: center teacher و temperature
۵۵	multi-crop و ویژگی متراکم DINO
۵۶	Twins Barlow و ماتریس همبستگی
۵۷	covariance invariance، variance، VICReg:
۵۸	فروپاشی کامل و بعدی
۵۹	spectrum و rank مؤثر
۶۰	صورت بندی modeling image masked
۶۱	BEiT و tokenizer گسسته
۶۲	MAE: encoder نامتقارن
۶۳	ratio mask و normalization target
۶۴	پایه سازی MAE کوچک
۶۵	iBOT و tokenizer آنلاین
۶۶	vec۲data و contextualized target
۶۷	I-JEPA و پیش بینی نهفته
۶۸	طراحی target و context در JEPA

۶۹	DINOv۲: داده‌گزینی و مقیاس
۷۰	tokenها register
۷۱	DINOv۳ و ویژگی بنیادی متراکم
۷۲	EVA-۰۲ و teacher هم‌تراز زبان
۷۳	CLIP و loss تصویر - متن
۷۴	SigLIP و loss pair sigmoid
۷۵	SigLIP ۲ و recipe ترکیبی
۷۶	kNN و probe linear
۷۷	fine-tuning و decay LR layer-wise
۷۸	prompt visual و adapter LoRA،
۷۹	انتقال متراکم و pyramid feature
۸۰	feature patch با correspondence
۸۱	کتابخانه PyTorch و SDPA
۸۲	torchvision و timm
۸۳	Transformers Face Hugging
۸۴	MMSelfSup و solo-learn Lightly،
۸۵	streaming و WebDataset
۸۶	all-gather DDP و batch مؤثر
۸۷	EMA و checkpoint توزیع شده
۸۸	precision mixed و پایداری عددی
۸۹	طراحی patch/temperature/mask ablation
۹۰	عیب‌یابی collapse
۹۱	ارزیابی چندمحوری نمایش
۹۲	مرزهای ۲۰۲۵-۲۰۲۶
۹۳	طراحی پروژه نهایی و گزارش پژوهشی
۹۴	مرور یکپارچه فصل و اتصال به فصل دوازدهم

جمع‌بندی فصل

این فصل نشان داد ترنسفورمر بینایی را نمی‌توان با جمله «تصویر را به وصله تقسیم و attention اعمال می‌کنیم» فهمید. توکن‌سازی یک bottleneck اطلاعاتی و محاسباتی است؛ encoding position هندسه را بازمی‌گرداند؛ attention یک kernel داده‌محور با هزینه و شرطی بودن خاص است؛ و معماری سلسله‌مراتبی یا کارآمد، گراف تعامل را محدود می‌کند.

در یادگیری خودنظارتی نیز نام روش‌ها مهم‌تر از سه سؤال بنیادی نیست: target چیست؟ چه ناوردایی ساخته می‌شود؟ و ضد فروپاشی چگونه عمل می‌کند؟ InfoNCE از، ها، DINO negative از معلم EMA، و VICReg centering از واریانس و کوواریانس، MAE از بازسازی target masked و I-JEPA از پیش‌بینی نهفته استفاده می‌کنند. مدل‌های بنیادی جدید این ها، objective را با داده‌گزینی،

مقیاس، وضوح بومی، distillation و هم ترازی متن ترکیب می کنند. دانشجوی مهندس حرفه‌ای باید بتواند هم فرمول را مشتق کند، هم path code را در PyTorch و کتابخانه‌های بالاتر ردیابی کند، هم collapse و leakage را تشخیص دهد، و هم مدل را با معیارهای جهانی، متراکم، robust و محاسباتی ارزیابی کند. فصل دوازدهم می‌تواند از این نمایش‌های بنیادی به مدل‌های مولد، vision-language diffusion یا تحلیل ویدئو وارد شود.

منابع منتخب فصل

Bibliography

- [1] A. Vaswani et al., “Attention Is All You Need,” *NeurIPS*, 2017.
- [2] A. Dosovitskiy et al., “An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale,” *ICLR*, 2021.
- [3] H. Touvron et al., “Training Data-Efficient Image Transformers and Distillation through Attention,” *ICML*, 2021.
- [4] W. Wang et al., “Pyramid Vision Transformer: A Versatile Backbone for Dense Prediction without Convolutions,” *ICCV*, 2021.
- [5] Z. Liu et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” *ICCV*, 2021.
- [6] H. Fan et al., “Multiscale Vision Transformers,” *ICCV*, 2021.
- [7] Y. Li et al., “MViTv2: Improved Multiscale Vision Transformers for Classification and Detection,” *CVPR*, 2022.
- [8] Z. Tu et al., “MaxViT: Multi-Axis Vision Transformer,” *ECCV*, 2022.
- [9] C. Ryali et al., “Hiera: A Hierarchical Vision Transformer without the Bells-and-Whistles,” *ICML*, 2023.
- [10] J. Su et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, 2024; arXiv:2104.09864.
- [11] T. Dao et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” *NeurIPS*, 2022.
- [12] T. Dao, “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning,” *ICLR*, 2024.
- [13] M. S. Ryoo et al., “TokenLearner: What Can 8 Learned Tokens Do for Images and Videos?,” *NeurIPS*, 2021.
- [14] Y. Rao et al., “DynamicViT: Efficient Vision Transformers with Dynamic Token Sparsification,” *NeurIPS*, 2021.

- [15] D. Bolya et al., “Token Merging: Your ViT but Faster,” *ICLR*, 2023.
- [16] M. Dehghani et al., “Patch n’ Pack: NaViT, a Vision Transformer for Any Aspect Ratio and Resolution,” *NeurIPS*, 2023.
- [17] L. Beyer et al., “FlexiViT: One Model for All Patch Sizes,” *CVPR*, 2023.
- [18] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” arXiv:1807.03748, 2018.
- [19] T. Chen et al., “A Simple Framework for Contrastive Learning of Visual Representations,” *ICML*, 2020.
- [20] K. He et al., “Momentum Contrast for Unsupervised Visual Representation Learning,” *CVPR*, 2020.
- [21] X. Chen, S. Xie, and K. He, “An Empirical Study of Training Self-Supervised Vision Transformers,” *ICCV*, 2021.
- [22] M. Caron et al., “Unsupervised Learning of Visual Features by Contrasting Cluster Assignments,” *NeurIPS*, 2020.
- [23] J.-B. Grill et al., “Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning,” *NeurIPS*, 2020.
- [24] X. Chen and K. He, “Exploring Simple Siamese Representation Learning,” *CVPR*, 2021.
- [25] M. Caron et al., “Emerging Properties in Self-Supervised Vision Transformers,” *ICCV*, 2021.
- [26] J. Zbontar et al., “Barlow Twins: Self-Supervised Learning via Redundancy Reduction,” *ICML*, 2021.
- [27] A. Bardes, J. Ponce, and Y. LeCun, “VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning,” *ICLR*, 2022.
- [28] T. Wang and P. Isola, “Understanding Contrastive Representation Learning through Alignment and Uniformity on the Hypersphere,” *ICML*, 2020.
- [29] H. Bao et al., “BEiT: BERT Pre-Training of Image Transformers,” *ICLR*, 2022.
- [30] K. He et al., “Masked Autoencoders Are Scalable Vision Learners,” *CVPR*, 2022.

- [31] Z. Xie et al., “SimMIM: A Simple Framework for Masked Image Modeling,” *CVPR*, 2022.
- [32] J. Zhou et al., “iBOT: Image BERT Pre-Training with Online Tokenizer,” *ICLR*, 2022.
- [33] A. Baevski et al., “data2vec: A General Framework for Self-Supervised Learning in Speech, Vision and Language,” *ICML*, 2022.
- [34] M. Assran et al., “Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture,” *CVPR*, 2023.
- [35] M. Oquab et al., “DINOv2: Learning Robust Visual Features without Supervision,” *Transactions on Machine Learning Research*, 2024.
- [36] T. Darcet et al., “Vision Transformers Need Registers,” *ICLR*, 2024.
- [37] O. Siméoni et al., “DINOv3,” arXiv:2508.10104, 2025.
- [38] Y. Fang et al., “EVA-02: A Visual Representation for Neon Genesis,” arXiv:2303.11331, 2023.
- [39] A. Radford et al., “Learning Transferable Visual Models from Natural Language Supervision,” *ICML*, 2021.
- [40] X. Zhai et al., “Sigmoid Loss for Language Image Pre-Training,” *ICCV*, 2023.
- [41] M. Tschannen et al., “SigLIP 2: Multilingual Vision-Language Encoders with Improved Semantic Understanding, Localization, and Dense Features,” arXiv:2502.14786, 2025.
- [42] S. Kornblith et al., “Similarity of Neural Network Representations Revisited,” *ICML*, 2019.
- [43] E. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” *ICLR*, 2022.
- [44] M. Jia et al., “Visual Prompt Tuning,” *ECCV*, 2022.
- [45] PyTorch Contributors, “Scaled Dot Product Attention and MultiheadAttention Documentation,” official documentation, accessed July 2026.
- [46] R. Wightman and Contributors, “PyTorch Image Models (timm) Documentation,” official documentation, accessed July 2026.

- [47] Hugging Face Contributors, “Transformers Vision Model Documentation: ViT, MAE, BEiT, DINO and I-JEPA,” official documentation, accessed July 2026.
- [48] Meta AI Research, “DINOv2 Reference PyTorch Implementation and Models,” official repository, accessed July 2026.
- [49] Meta AI Research, “DINOv3 Reference PyTorch Implementation and Models,” official repository, accessed July 2026.
- [50] Meta AI Research, “Masked Autoencoders Official PyTorch Implementation,” official repository, accessed July 2026.
- [51] Lightly AI, “LightlySSL Documentation and Self-Supervised Learning Methods,” official documentation, accessed July 2026.
- [52] OpenMMLab Contributors, “MMSelfSup: OpenMMLab Self-Supervised Learning Toolbox,” official repository and documentation, accessed July 2026.
- [53] V. G. T. da Costa et al., “solo-learn: A Library of Self-Supervised Methods for Visual Representation Learning,” *JMLR*, 2022.
- [54] J. K. K. Contributors, “WebDataset: PyTorch I/O for Large Scale Visual Data,” official project documentation, accessed July 2026.

پیوست آ

راهنمای ساخت، اجرا و بازتولید

آ.۱ ساخت PDF

```
1 xelatex image_processing_next_decade_ch11_fa.tex
2 xelatex image_processing_next_decade_ch11_fa.tex
```

اجرای دوم برای تثبیت فهرست مطالب و ارجاع‌های داخلی است.

آ.۲ ساخت محیط پایتون

```
1 python -m venv .venv
2 source .venv/bin/activate          # Windows: .venv\Scripts\activate
3 python -m pip install --upgrade pip
4 python -m pip install -r requirements.txt
```

بسته‌های سنگین مانند `timm`، `transformers` و `lightly` برای `test smoke` پایه ضروری نیستند؛
کدهای آموزشی تنها به `PyTorch` نیاز دارند.

آ.۳ تولید شکل‌ها

```
1 python generate_figures.py
```

شکل‌ها مفهومی و بازتولیدپذیرند. نمودارهایی که داده مصنوعی دارند در `caption` به‌صراحت آموزشی معرفی شده‌اند.

آ.۴ اجرای `test smoke`

```

1 python code/patchify.py
2 python code/minimal_vit.py
3 python code/info_nce.py
4 python code/simclr_minimal.py
5 python code/mae_minimal.py
6 python code/dino_ema.py
7 python code/native_resolution_pack.py
8 python code/benchmark_attention.py --tokens 64

```

آ.۵ نمونه استفاده از مدل کتابخانه‌ای

```

1 python -m pip install timm
2 python code/inspect_pretrained.py

```

این مثال به صورت پیش فرض وزن دانلود نمی کند. برای وزن پیش آموزته، pretrained=True و دسترسی به checkpoint لازم است؛ نام مدل و preprocessing config باید ثبت شود.

آ.۶ ساخت manifest و checksum

```

1 python - <<'PY'
2 from pathlib import Path
3 import hashlib
4 root = Path('.')
5 with open('manifest_sha256.txt', 'w', encoding='utf-8') as f:
6     for p in sorted(x for x in root.rglob('*') if x.is_file()):
7         if p.name == 'manifest_sha256.txt':
8             continue
9         h = hashlib.sha256(p.read_bytes()).hexdigest()
10        f.write(f'{h} {p.as_posix()}\n')
11 PY

```

آ.۷ پیش پرواز علمی

پیش از تحویل نتیجه پژوهشی، موارد زیر بررسی شود:

۰۱. اعداد نمودار از فایل log/CSV تولید شوند؛
۰۲. داده مصنوعی با برچسب illustrative مشخص باشد؛
۰۳. split و overlap با pretraining مستند شود؛
۰۴. commit checkpoint و license مشخص باشد؛

۵. مدل در چند seed و با فاصله اطمینان ارزیابی شود؛
۶. benchmark سرعت روی سخت افزار و نسخه نرم افزار هدف تکرار شود؛
۷. case failure و گروه های ضعیف پنهان نشوند.